Towards Statement-Level Decompiler Evaluation

A Framework for Evaluating Decompilers at Statement-Level Granularity

A thesis submitted in partial fulfillment of the requirements for a bachelor degree in Computer Science

John Chen 851585377

Open Universiteit, The Netherlands

***Abstract*—There are times software needs to be inspected through its source code, but the source code may not always be accessible. This can happen when legacy software needs updating or in the case of security events where malware needs to be investigated. In order the mitigate or prevent the possible costs of such events, time is dedicated to understand and recover readable forms of the corresponding lost or unknown code. This process called reverse engineering uses decompilation to reconstruct the workings of the original source code. Better decompilation helps developers to understand, recover and design software. Improving decompilation is an ongoing endeavour, where developers use oracles to evaluate their results. Each oracle helps guide the development of a different decompiler aspect. Next to semantic correctness, decompilation quality is important to improve reverse engineering success. We propose a proof of concept that expands on statement-level marker instrumentation to inspect quality aspects through individual statements in code. Based on the marker mechanism, we showcase how to evaluate decompilation performance and how results can be used to guide decompiler development.**

***Index Terms*—Software Engineering, Compilation, Decompilation, Codemarkers, Instrumentation, Benchmarking.**

I. Introduction

A compiler transforms the source code to executable machine code. During compilation to machine code, loss of human-level context and abstraction takes place. Decompilers are tools that try to recover source code from the machine binary. Decompilers translate machinecode to a higher code language and tries to recover as much of the high-level context as possible in the process. Thus the compilation and decompilation cycle is a process that incurs loss of information, a problem still unsolved. The current state of decompilation involve tools that require human intervention. Modern decompilers; e.g. ANGR [1], Ghidra [2], Hexrays [3], Binary Ninja [4], produce pseudo-code variants to indicate uncertainty pertaining to types and procedures. Large Language Model (LLM)-integrated decompilers try to improve upon the readability of such decompiled source code [5]. In order to improve decompilation, researchers have examined aspects like readability and recompilability [6], [7]. The aspect readability does not have a singular definition, leading to different interpretations for ideal decompiler output, from exact reproduction to outputting readable and semantically valid source code.

Instances of direct correspondence of executed machine code to source are available through tools like DWARF [8] that provide debug information, in literature DWARF is the bedrock of numerous solutions that aid decompilation research. Contemporary decompilation artifacts use debug hints, and in some cases supplementary augmentations, to extract and cross-reference function-level code with the end goal of accurately determining the corresponding sourcecode and control flow.

In this paper we propose a proof of concept that builds upon the marker concept as an oracle on statement-level detail. In our proof of concept we use a variant of codemarkers to instrument source code, that allows mapping between precompilation and postdecompilation statements. We showcase for a decompiler how its performance on code can be benchmarked, compared across multiple codebases and how these findings can be used to improve decompilers.

II. Related work

Theodoridis et al. [9] worked with markers on source code instrumentation for decompilation to detect deadcode-elimination within a custom pipeline. These markers are instrumented without body to preserve the markers through the (de)compilation pipeline [10], [11]. Markers can detect the absence and presence of statements under certain conditions.

A similar principe is applied by Deb'm [12] with the addition of modifying the compilation process, hence shifting the usability of markers towards a more direct benchmarking purpose of construct recovery. Deb'm instruments its own custom generated source code to benchmark (de)compilation. They use intermediate-representation and binary for identification of constructs, contrasting to our usage of the Clang toolset.

These markers persist through the modified (de)compilation process and map between the original and decompiled source.

For real-world code, function-level extraction procedures are some of the utilized methods to map functions as isolated pairs to its in-source counterparts, assess real-world code [13], [14], and train LLM-based decompilers [5].

Comparable instrumentations are also applied outside the source code format, Pang et al. [15] developed OracleGT to insert label-like directives into assembly code. OracleGT then intercepts the instrumented assembly file before its input into the GNU assembler to gather oracle data.

Ying Cao et al. [16] measured the performances of various decompilers using the approaches; recompilability, semantic consistency, readability and human study. The decompilers achieved varying results, with relative low similarity scores,

where similarity is quantified by the edit distance between IR and source. Ying Cao et al. also utilize markers as a method to enable control path identification. Demonstrating that markers can be used for mapping control paths. Furthermore they identify that the static analysis phase remains the primary factor that limits decompilation quality, an aspect that markers are suited to evaluate.

Campbell [6] explores the concept of readability and posits that cyclomatic complexity does not fairly assess the quality readability, she showcases an alternative measuring algorithm and juxtaposes them. Her findings can be used to assess the recovery of decision statements, where decompilers can recover the result as multiple sequential if statements instead.

Eom et al. [7] studies a readability metric based on selected features they extract through an abstract syntax tree (AST) of decompiled code. They utilize surveys to finetune the results and calculate metrics over the features. Their extraction of features is similar to our methodology, using the AST to determine the findings. Thus their feature methodology shows strong compatibility with the marker concept for quality analysis.

III. Measuring Decompilation

A. The role of artificial and real-world code in (de)compiler development.

Artificial code from codegenerators is ubiquitous in the development and testing pipelines of decompiler developers. Historically for the decompiler's counterpart, the compiler, literature shows that the sole use of artificial code and fixed testing suites are insufficient to accurately assess compiler performance [17]. For languages like C, which allow undefined behaviour, real-world applications contain types of code that codegenerators choose to avoid [17], [18]. Codegenerators like DSmith used in the decompiler development for performance analysis[16] show that extending the range of possible code is beneficial to decompiler development.

It is therefore important to expose decompilation tools to a wide range of code as possible. However even if we have a statement to statement mapping, semantic analysis is necessary to assess equivalence. Determining semantic equivalence between code in general is undecidable [19], but decompilers can approximate the original characteristics of the code using heuristics.

In our tests we include source code generated by codegenerators, which we label as "artificial". We also include source code that is written by humans, which we label as "realworld". For this proof of concept we randomly selected maintained codebases to instrument and build. Based on the patterns of preliminary results, variants will be included to survey for other possible effects and to show ways how variants can be assessed.

B. Instrumentation mechanism

The callee function of the inserted calls alongside other function declarations found in the source are marked on a best-effort base with attributes to disable optimization using the Clang AST recursive visitor, this ensures the markers



Fig. 1: Instrumented CSmith codebase with seed=3. Markers persisting through (de)compilation of C source using Ghidra.

persists. However the presence of a macro can make the instrumentation result in an invalid syntax. To mitigate this, we implemented checks that inspect the site of insertion for the presence of such macros. If these macro's are present, we skip the insertion. For markers we check the presence of enclosing brackets for statements, also known as compound statements in the Clang AST. These compound statements only allow the nodes of other Clang statement classes, ensuring safe insertion except in the presence of macro's. For each codebase we explicitly mark the codebase source code, but exclude external libraries and their instrumentation, so that the assessable statements relate more specifically to the codebase we want to compare.

Each marker marks its "markee", which is the statement following it. We then traverse the AST for both the source precompilation and postdecompilation to extract findings. We can then map these statements by crossreferencing the recorded marker indexes. The application of these modifications allow the eventual result of cross-compilation mapping seen in Figure 1.

The artifact developed for the instrumentation, Mage, implements the Clang AST visitor [20] to interpret and traverse (pseudo) C code, where it can systematically visit each statement and declaration.

Currently Mage targets the following Clang statement classes [21] for instrumentation:

1) `CaseStmt`, represents case statements within a switch structure.
2) `CallExpr`, represents a function call.
3) `DeclStmt`, represents a declaration.
4) `ForStmt`, represents a for-loop structure.
5) `GotoStmt`, represents a goto jump statement.
6) `IfStmt`, represents a conditional if statement.
7) `SwitchStmt`, represents a switch strucure.
8) `WhileStmt`, represents a while-loop structure.

The statement list is not exhaustive and is implemented to demonstrate this proof of concept.

C. Research pipeline

We use the following pipeline, where the phases ”Codebase” up to and including ”Build”, proceed as per their build instruction for repositories or series of Clang invocations for artificial code. However we do require a minimum of name preservation to be able to measure decompilation performance, so this adjustment using compiler flags is added when necessary. For each pipeline instance and phase we have configuration files that record the exact procedure that is performed to continue pipelines artifacts from one phase to another.

- Codebase
  Consists of storing code for assessment which can include: any arbitrary repository, custom or artificially generated code.
- Instrumentation
  The instrumentation phase makes a copy of some selected codebase and instruments this codebase using ”Mage”, resulting in instrumented codebases.
- Build
  Instrumented code gets built according to its accompanying repository instructions, if necessary with the name preservation compiler flags. For artificial code, we invoke Clang as follows:
  After the build completes, all the produced binaries are moved to a designated directory ready for decompilation.
- Decompilation
  We select a decompilation script that contains the build instruction of the repository to decompile the instrumented code and extract the produced binaries.
- Eval
  After the code is decompiled we extract findings from the markers, each finding has a similar format to what is depicted in Listing 1.
  For each finding sharing the same marker index, we perform a comparison, which produces a result like Listing 2 for each mapping. After all the findings are compared we produce a summary, for an example see appendix A, where we sum up the amount of instances the original construct was able to be recovered and to what degree the attributes were restored.
- Analysis
  We aggregate the information found in the ”Eval”-phase under common characteristics. For each of these aggregations the probability of construct and general recovery is calculated. Finally we perform cross group examinations and compare their results.


```
"2": {
    "decom": [
      {
        "ASTNode": "IfStmt",
        "Source": "... /3.c (489, 3) ",
        "properties": {
          "cond_expr_types": [
            {
              "is_const": false,
              "is_pointer": false,
              "type": "_Bool"
            }
          ],
          "cond_stmt_class": "BinaryOperator",
          "has_cond_expr": true,
          "has_cond_var": false,
          "has_else": false,
          "has_markee": true
        }
      }
    ],
    "oracle": [
      {
        "ASTNode": "IfStmt",
        "Source": "... /3.c (62, 5) ",
        "properties": {
          "cond_expr_types": [
            {
              "is_const": false,
              "is_pointer": false,
              "type": "_Bool"
            }
          ],
          "cond_stmt_class": "BinaryOperator",
          "has_cond_expr": true,
          "has_cond_var": false,
          "has_else": false,
          "has_markee": true
        }
      }
    ]
  }
```


Listing 1: Single finding for the marker with index 2 in Figure 1.


```
"2": [
    {
      "markee_construct": "IfStmt",
      "parity_cond_expr_types": true,
      "parity_cond_stmt_class": true,
      "parity_cond_var_stmt_class": true,
      "parity_cond_var_types": true,
      "parity_has_cond_expr": true,
      "parity_has_cond_var": true,
      "parity_has_else": true,
      "recovered_construct": true
    }
]
```


Listing 2: An evaluation finding of the marker with index 2 mapping instance for Figure 1 using the results in Figure 1.

D. Benchmark methodology

For each of the statements we assess the recovery in terms of the enclosing Clang class abstractions, i.e. Clang statement classes. The chosen method is the most direct form of assessment, which does not make extra assumptions about how to process the findings. We focus on the construct (statement class of the markee) recovery aspect as follows, i.e. did the decompiler manage to restore the statement class to such an extent that the clang parser can recognize and compare it the original source. The Clang parser can handle errors in the decompiler code, has the capability to decide if the processed syntax is compilable and allows us to access Clang's tested arsenal of methods for handling statements. However findings gathered through the Clang AST walker can be dependent on the included header libraries. Certain statements have the requirement of needing the right headers to resolve completely or else the precompilation and postdecompilation mappings are found as `RecoveryExpr`. When this happens

these particular coupled codebase-library statements cannot be evaluated completely, these instances are evaluated as a recovery loss.

Evaluating library inclusions requires the instrumentation of the libraries, which is a group of evaluation on its own as the inclusion of the same libraries can occur in multiple projects effecting the results. Introducing the evaluation of libraries within the codebase also reduces the independence of code evaluation. The AST-based findings are gathered without the explicit inclusion of any library, this has the effect of concentrating the comparable results to in source statements with no external dependencies. For the decompiler findings we choose Ghidra for the showcase.

We want to see if there is any difference in the performance of a decompiler between "realworld" and "artificial" code. In order to benchmark a particular instance of comparison within this subject we use the following format: benchmark(any, real-world, algorithmic, ghidra)

- code-intent, the intent the code is produced or generated with, commonly declared in the readme of repositories or related indexed tags, e.g. as used by GitHub.
- code-type, synthetic or real-world; produced by humans or algorithmically.
- decompiler-type, algorithmic or ml-based decompilation. For the demonstration we showcase results from algorithmic decompilation.
- decompiler, the decompiler that is applied with some setting.

For this showcase we make three groups, one containing "artificial" code i.e. code generated by dedicated codegenerators like CSmith, another group "realworld" consisting of codebases written by humans and "aggregated" that includes codebases from both:

- Group "realworld" (appendix XVIII).
- Group "artificial" (appendix XIX).
- Group "aggregated" (appendix XX).

We compiled the codebases using their repository instructions and otherwise with the clang compiler. If the build process needed modifications to preserve names, then these flags were passed to the build tool, the specific details are recorded in the configurations. We then inspect each constructs and interpret the results.

E. Comparison methodology

At the end of the eval-phase we aggregate the results of the codebase summaries within a benchmark group to a collective summary of the group, e.g. benchmark(any, artificial, algorithmic, ghidra).

We use each codebase summary to examine if the decompiler performance differed:

1) from codebase to codebase.
2) from the aggregated artificial to the realworld benchmark group.

For algorithmic decompilers, the used decompiler algorithm solves a certain proportion of the population of possible code instances, therefore we can interpret this proportion in the context of a bernouilli distribution.

This gives us, at least, the following options to choose, if we want to see if performance is "comparable":

- Two-proportion Z-Test [22]. This is used to check if the proportions of the involved samples are similar. The test is suitable for larger proportions and has the requirements:
  1) Binary outcomes: outcomes are either true or false.
  2) Independent random samples, where one sample does not exert influence on the other.
  3) Success or failure condition:

$$\begin{aligned} n_1\hat{p}_1 &\geq 10 \quad \text{(Group 1 Successes)} \\ n_1(1-\hat{p}_1) &\geq 10 \quad \text{(Group 1 Failures)} \\ n_2\hat{p}_2 &\geq 10 \quad \text{(Group 2 Successes)} \\ n_2(1-\hat{p}_2) &\geq 10 \quad \text{(Group 2 Failures)} \end{aligned}$$

- Fisher's Exact Test [23]. This is used to check if the proportions of the involved samples are similar. Fisher exact test is used for small sample sizes and when there are occurences of "cells", i.e. events, that have less than 5 counts.
- Pearson's Chi-square Test [24] A test that checks how likely it is that some given two proportions are independent of each other.
- Likelihood Ratio Test [25] A test that checks if the number of observations in some category fits a theoretical expectation.

Given that we are measuring numerous individual statements and based on preliminary findings, we expect the samples not to fullfill Fisher's Exact Test requirements for small samples. We assume that the code content of the codebases are independent of each other. The theoretical probability for ghidra's algorithm is unknown and we want to compare performances, this leaves us the two-proportion z-test.

Within each group we calculate the probabilities of recovery as follows:

$$\hat{p}_i = \frac{recoveredConstruct_i}{viableComparison_i}$$

$\hat{p}_i$, the recovery probability of sample i, is taken to be representative of the Bernouilli distribution of the sample. Viable comparisons are possible oracle and decompilation pairs, as seen in findings for an index that we can evaluate (Fig 1). One oracle can map to multiple decompilation occurences, this can occur if source code is used to build multiple binaries.

An alternative but unviable consideration is to take the recovery over the total inserted markers. As codebases can reuse source code files to produce multiple binaries within a build, the effect is that each oracle marker can have more than 1 comparison (Listing 3).

```
"7": {
  "decom": [
    {"ASTNode": "IfStmt", ...},
    {"ASTNode": "BinaryOperator", ...},
    {"ASTNode": "RecoveryExpr", ...},
  ],
  "oracle": [
    {"ASTNode": "IfStmt", ...},
    }
  ]
}
```

Listing 3: 1 marker and 3 possible comparisons over multiple binaries.

A hypothetical situation with each marker having 3 comparions and using the oracle marker total instead, would erroneously triple the probability of the calculation here. Therefore viable comparisons is more accurate in assessing the recovery rate.

From the groups we make pairs that we compare using the selected "Two Proportion Z-Test", where each group is only eligible for comparison if they satisfy the test's accompanying requirements.

We assume the standard test hypotheses:

1) Null hypothesis; $\hat{p_1} = \hat{p_2}$, i.e. the decompilation success in terms of statement recovery for this pair is similar.
2) Alternative hypothesis; $\hat{p_1} \neq \hat{p_2}$.

Thus if we group the codebases by either their artificial or real-world origin, we can make an assessment based on the outcomes of the Z-test. We make no assumptions in favor of the null-hypothesis, i.e. that the decompilation performance is similar for the compared samples. This assumption brings us to the unpooled variant to calculate the standard error (SE):

$$SE_{\text{unpooled}} = \sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_2(1-\hat{p}_2)}{n_2}} \tag{1}$$

The comparison of codebases can give us false positives and false negatives. where false negatives are more common with small sample sizes [26]. To deal with these errors, there are multiple options [27] we can choose. For the purpose of the showcase we limit our options to the following:

- Bonferroni [28], requirements: none.
- Tukey's Honestly Significant Difference [29], requirements:
  1) "observations are independent within and among groups."
  2) "observations within each group are normally distributed."
  3) "distributions from which the samples are drawn have the same finite variance."
- Holm-Bonferroni [30], a more "powerful" version of the Bonferroni, should be chosen over the normal Bonferroni.
- Dunnett [31], main requirements: multiple samples are compared to a single control sample.
- Sidak [32], the sidak test is more accurate compared to the normal Bonferroni when applied to independent samples. If the samples are not independent then it overestimates the error rate.

Because we use a Bernouilli model and compare codebases with each other, Tukey and Dunnet are out of the question. Guillermou et al. [33] compared corrections and the outcomes, finding Holm-Bonferroni to be "favorably" balanced of the methods. Of both Holm-Bonferroni and Sidak, Holm-Bonferroni was the confirmatory method to be found more powerful, so we select Holm-Bonferroni as the correction method. Furthermore we select $\alpha = 0.05$ as the baseline significance level.

Finally we interpret the results for the groups.

IV. Results

We gathered findings from 3 generators for artificial code; CSmith [17], DSmith [34] and Yarpgen [18]. With each of these generators we used the seeds from 0 to 1,000 to generate source code with no extra flags, except for Yarpgen where we exclude the random seed index 0. We then compile each of these using Clang with no flags, giving us a total of roughly 265,000 viable mappings to compare.

For real-world code we installed the required build packages if needed and compiled these using their repository instructions. This resulted in about 10 million viable comparisons, where one repository alone was responsible for 9.7 million comparisons. Some samples were also compiled on variants with minor characteristics to survey for possible effects using Clang target flags. For each codebase, the build commands are recorded within their respective configurations in the pipeline.

A. Evaluating real-world code and interpreting results

The following section uses appendix C-A. We first investigate why certain constructs are recovered frequently as other constructs. Then we inspect the recovered constructs on the attributes and finally we interpret the results using the found observations.

`CallExpr`'s prominent alternative recovery instances are: `BinaryOperator`, `IfStmt` and the Clang abstraction `RecoveryExpr` for pseudocode. Complex statements, e.g. expression class category, can cause recovery to manifest into flattened representations of its constituents. For some `CallExpr`, the presence of operators in arguments can therefore lead to the decomposition in Listing 4.

```
// oracle, post increment passes value before update
mage_alpha17476(17476);
hc_asprintf(
  &hiprtc_options[hiprtc_options_idx++],
  "-D INCLUDE_PATH=%s", folder_config->cpath_real);
// decom, lVar6 takes place of preincrement value.
mage_alpha17476(17476);
lVar6 = (long)hiprtc_options_idx;
hiprtc_options_idx = hiprtc_options_idx + 1;
hc_asprintf(
  hiprtc_options + lVar6,
  "-D INCLUDE_PATH=%s", folder_config->cpath_real);
```

Listing 4: Recovery of `CallExpr` to `BinaryOperator` and "++"-operator extraction in the decompilation result.

Depending on the interpretation of software engineering (SE) principles, this flattening by the decompiler could be

considered a quality regression. `CallExpr` restorations to `RecoveryExpr` can look peculiar, but the recovery abstraction tells us that semantic information is lacking to fully determine the Clang abstraction form, e.g. Listing 5. The spelled tokens are preserved and either `hc_dlclose(...)` or `bridge_ctx` is missing semantic information.

```
// oracle
void ... bridge_unload(bridge_ctx_t *bridge_ctx) {
mage_alpha18389(18389);
if (bridge_ctx->bridge_handle) {
  mage_alpha18390(18390);
  hc_dlclose(bridge_ctx->bridge_handle);}
}
// decom
void bridge_unload(bridge_ctx_t *bridge_ctx) {
bridge_ctx_t *bridge_ctx_local;
mage_alpha18389(0x47d5);
if (bridge_ctx->bridge_handle != (hc_dynlib_t)0x0) {
  mage_alpha18390(0x47d6);
  hc_dlclose(bridge_ctx->bridge_handle);
} return;}
```

Listing 5: Recovery of `CallExpr` to `RecoveryExpr` for markee of 18390, because of missing semantic information.

When `CallExpr` get recovered as `IfStmt`, the same extraction to separate variables can happen for the expressions in the arguments (Listing 6).

```
// oracle
screen_write_putc(ctx, &gc, bottom ? 'v' : 'x');
// decom
if (bottom == 0) {uVar2 = 'x';}
else {uVar2 = 'v';}
screen_write_putc(ctx,&gc,uVar2);
```

Listing 6: Extraction of the `BinaryOperater` arguments in decompilation result.

For `IfStmt` we find that `BinaryOperator` extractions can happen even while semantic information is missing, this has as consequence that some `BinaryOperator` will be found by Clang as a `RecoveryExpr` instead.

For `CaseStmt` the recovery is most often found as either a `GotoStmt` or `LabelStmt`, suggesting that Ghidra has not managed to interpret these components as part of a Switch completely. Notably the C grammar allows statements to be on the same scope as the case statements, which we use to instrument `CaseStmt`. The statements on case level will execute due to fallthrough (Listing 7).

```
switch (x) {
    mage_alpha(83);
    case 1:
        printf("One\n");
        break;
    mage_alpha(84);
    case 2: printf("Two\n"); // fallthrough
    mage_alpha(85);
    ...
}
```

Listing 7: Case statements with other statements on the same scope can execute through fallthrough. If we replace marker 85 with a print statement, it will print at the end of case 2.

This legal edge case can contribute to its low recovery if developers have not adapted their decompiler performances for it. The majority of case statements and their markers are not recovered in the tested Ghidra version.

Switch statements are most often recovered as `BinaryOperators` and `IfStmt`. If decompilers are unable to unify their interpretation of individual case statements to a single switch, then manifestations as individual recovery forms appear in the form of sequential `IfStmt`. For `IfStmt` that are (deeply) nested, its decompilation can appear as a flattened sequence of guarding if statements that return immediately.

`DeclStmt` seems to show close to zero direct recovery, however the following reveals that `DeclStmt` without complete type information tend to show up as `BinaryOperators` with the "=" operator. The recovered `DeclStmt` were all single declarations.

```
// Decom:
hd = offset + hdrsize;
llen = linelen(buffer + hd,size - (long)hd);
// Oracle:
int hd = hdrsize + offset;
unsigned long llen = linelen(buffer + hd, size - hd);
```

Listing 8: `DeclStmt` typeloss leads to pseudocode `BinaryOperator` interpretation.

For recoveries as `CallExpr` and `BinaryOperator` instances, loss of markers and rearrangements occur, leading to detections of markers as `CallExpr` (Listing 9). For `CallExpr` to `BinaryOperator`, the same extraction occurs as with `IfStmt`.

```
// oracle
struct screen *s = ctx->s;
mage_alpha9582(9582);
struct grid_cell gc;
mage_alpha9583(9583);
u_int cx, cy, i;
cx = s->cx;
cy = s->cy; ...
// decom
mage_alpha9581(0x256d);
psVar1 = ctx->s;
mage_alpha9582(0x256e);
mage_alpha9583(0x256f);
cx_00 = psVar1->cx;
cy_00 = psVar1->cy; ...
```

Listing 9: Decompilation is missing type information and declarations are rearranged. Marker 9582 detects 9583 instead, 9583 detects `BinaryOperator` instead of a `DeclStmt`

Having specific information allows decompilation to manifest `DeclStmt` as `CallExpr` as seen in Listing 10.

```
// oracle
mage_alpha5109(5109);
char c, last[] = "X;:", **argv, *value; ...
// decom
mage_alpha5109(0x13f5);
builtin_strncpy(last,"X;:",4); ...
```

Listing 10: Ghidra decompilation shows transformation of declarations to function call.

Ghidra decompiled the instructions to a builtin pseudo function, these pseudo function can have corresponding counterparts in libraries, strncpy is found in `<string.h>`.

`ForStmt` and `WhileStmt` recover each other as the most numerous alternative loop structure. Recoveries of `WhileStmt` and `ForStmt` to do-while structures have manifestations that follow comparable extractions leading to `BinaryOperator` (Listing 11).

```
// oracle 18512
void ... dictstat_read(hashcat_ctx_t *hashcat_ctx) {
  ...
  mage_alpha18512(18512);
  while (!hc_feof(&fp)) {
    dictstat_t d;
    const size_t nread =
      hc_fread(&d,sizeof(dictstat_t), 1,&fp);
    mage_alpha18513(18513);
    if (nread == 0) {continue;}
    lsearch(
      &d, dictstat_ctx->base, &dictstat_ctx->cnt,
      sizeof(dictstat_t), sort_by_dictstat
    );
    if (dictstat_ctx->cnt == MAX_DICTSTAT) {
      event_log_error(hashcat_ctx, "...",
      dictstat_ctx->filename);
      break;
    } ...
  }
}
// decom 18512
void dictstat_read(hashcat_ctx_t *hashcat_ctx) {
  ...
  mage_alpha18512(0x4850);
  do {
    iVar3 = hc_feof(&fp);
    if (iVar3 != 0) goto LAB_004dd131;
    sVar4 = hc_fread(&d,0x128,1,&fp);
    mage_alpha18513(0x4851);}
  while ((sVar4 == 0) || (lsearch(
      &d, pdVar1->base, &pdVar1->cnt,
      0x128, sort_by_dictstat),
      pdVar1->cnt != 100000));
  event_log_error(hashcat_ctx, "...",
    pdVar1->filename
  );
  } ...
}
```

Listing 11: Ghidra moves while-body statements to do-body in decompilations, expressions are assigned to separate variables first instead of passing directly and the control clause of the loop has its condition modified.

```
// oracle
mage_alpha18514(18514);
if (hc_lockfile(&fp) == -1) {
  hc_fclose(&fp);
  event_log_error(
    hashcat_ctx, "%s: %s",
    dictstat_ctx->filename, strerror(errno)); // <--
return -1; ...
// decom
mage_alpha18514(0x4852);
iVar3 = hc_lockfile(&fp);
if (iVar3 == -1) {
  hc_fclose(&fp);
  piVar4 = __errno_location(); // <-- extraction
  pcVar5 = strerror(*piVar4); // <-- extraction
  event_log_error(
    hashcat_ctx, "%s: %s",
    pdVar1->filename, pcVar5);
  iVar3 = -1;}
else {...} ...
```

Listing 12: Extraction of expressions to variables.

For complex control statements, rearrangements can effect the way the control clause evaluate. Another consequence is that markers that are part of the complex statement lose their markee, e.g. marker 18513 (Listing 11). `IfStmt` recovery to `BinaryOperator`, also involves extraction of operands to separate declarations (Listing 12).

A closer look at the recovery details shows Ghidra's lowest successes are in determining the exact types of the arguments and parameters. The name preservation is due to the inclusion of available debug information, which cannot normally be assumed. The presence of the callee en variadicity is determined through Clang's builtin methods, showing high success.

TABLE I: Realworld: Normalized proportion of recovered attributes for CallExpr, (val = attribute / recovered_construct).

Attribute	Proportion
name_preserved	0.7490
parity_arg_types	0.5958
parity_args_count	0.9986
parity_has_callee	0.7496
parity_param_types	0.3248
parity_params_count	0.4328
parity_side_effects	1.0000
parity_variadic	0.7138

The 3 largest recovered control constructs show the following regarding the recovery of their attributes (Table II):

TABLE II: Normalized proportion of recovered attribute parity, a dash means that the attribute does not apply.

Attribute	WhileStmt	IfStmt	ForStmt
parity_cond_expr_types	0.4852	0.7556	0.7657
parity_cond_stmt_class	0.1971	0.4718	0.7015
parity_has_cond_expr	1.0000	1.0000	1.0000
parity_has_else	–	0.8342	–
parity_has_incr	–	–	0.9961
parity_has_init	–	–	0.9752
parity_incr_expr_types	–	–	0.9278
parity_incr_stmt_class	–	–	0.1955
parity_init_stmt_class	–	–	0.9295
parity_init_types	–	–	0.9214

The control phases of these statements have varying degrees of involvement regarding program context in the order: initialization, condition and increment. Ghidra scored well on recognizing the presence of the control clauses with all scores over 97% for `has_init`, `has_cond_expr` and `has_incr`. The condition and increment expression types match ranged from 48% to 92% and the corresponding statement classes showed large variance. The condition clause statement class was recovered in incrementing order of success: incr, cond and init, suggesting larger difficulty due to semantic context.

As we've seen in the listings, extraction is a recurring method that is utilized by Ghidra, which the decompiled `ForStmt` accomodates with additional declarations and the `DoStmt` absorbs with its do-block from the `WhileStmt`. Within these control statement, the inclusion of other control statements can lead to the modification of the clause of the outer statement.

Recovery of the attributes for `WhileStmt`, `IfStmt` and `ForStmt` deteriorated with specificity. Overall Ghidra scored less than half on statement class recoveries (appendix C-A).

B. Evaluating artificial code and its comparison to realworld

The measurements from table XV, XVI and XVII are extracted from benchmark(any, artificial, algorithmic, ghidra).

For artificial code we can observe in V that Ghidra performs best on the `ForStmt`, `IfStmt`, `WhileStmt` and `CallExpr` constructs. The codegenerators did not include any `GoToStmt` or `SwitchStmt` within their outputs.

Inspecting the details for `CallExpr` for artificial code, brings the following (Table III).

TABLE III: Artificial: normalized proportion of recovered attributes for CallExpr, (val = attribute / recovered_construct).

Attribute	Proportion
name_preserved	0.8900
parity_arg_types	0.0614
parity_args_count	0.5194
parity_has_callee	0.8900
parity_param_types	0.0429
parity_params_count	0.3453
parity_side_effects	1.0000
parity_variadic	0.8900

Putting them side-by-side shows higher recovery for the realworld set on most attributes (Table IV). Recovery in particular related to arguments show a difference of roughly 50%, showing that realworld code has characteristics that benefit recovery of argument attributes.

TABLE IV: delta = “realworld” - “artificial” for CallExpr attribute recovery.

Attribute	delta
name_preserved	−0.141,0
parity_arg_types	0.534,4
parity_args_count	0.479,2
parity_has_callee	−0.140,4
parity_param_types	0.281,9
parity_params_count	0.087,5
parity_side_effects	0.000,0
parity_variadic	−0.176,2

Inspecting the attributes for the same top three control structures (Table V), shows the same pattern for the recovery order of statement classes as with realworld code; incr, cond and init.

TABLE V: Normalized proportion of recovered attribute parity, a dash means that the attribute does not apply.

Attribute	WhileStmt	IfStmt	ForStmt
parity_cond_expr_types	0.7940	0.8976	1.0000
parity_cond_stmt_class	0.7940	0.2631	0.8073
parity_has_cond_expr	1.0000	1.0000	1.0000
parity_has_else	–	0.9664	–
parity_has_incr	–	–	0.9927
parity_has_init	–	–	0.9964
parity_incr_expr_types	–	–	0.8671
parity_incr_stmt_class	–	–	0.0273
parity_init_stmt_class	–	–	0.9455
parity_init_types	–	–	0.8686

The while condition statement class recoveries show notable differences (Table VI). Currently CSmith and Yarpgen do not generate while-loops, nor do they possess flags that enable them. DSmith is the only one in the group that currently generates while-loops, skewing the result if it happens to form statements that Ghidra has relatively little trouble with. However DSmith does not generate `ForStmt` with default options. The codegenerators generated no `CaseStmt`, `SwitchStmt` and practically no `GotoStmt`. Their recoveries are as follows, set next to each other (Table VII).

TABLE VI: delta = ”real-world” - ”artificial”, a dash means that the attribute does not apply.

Attribute	WhileStmt	IfStmt	ForStmt
parity_cond_expr_types	0.308,8	0.142,0	0.234,3
parity_cond_stmt_class	0.596,9	−0.208,7	0.105,8
parity_has_cond_expr	0.000,0	0.000,0	0.000,0
parity_has_else	–	0.132,2	–
parity_has_incr	–	–	−0.003,4
parity_has_init	–	–	0.021,2
parity_incr_expr_types	–	–	−0.060,7
parity_incr_stmt_class	–	–	−0.168,2
parity_init_stmt_class	–	–	0.016,0
parity_init_types	–	–	−0.052,8

TABLE VII: Recovery comparison between artificial and real-world benchmarks. delta = ”artificial - ”realworld”.

Construct	Artificial	Real-world	delta
CallExpr	0.841,9	0.880,8	−0.038,9
ForStmt	0.608,5	0.629,4	−0.020,9
GotoStmt	0.000,0	0.836,8	−0.836,8
IfStmt	0.828,7	0.715,9	0.112,8
WhileStmt	0.795,1	0.464,0	0.331,1
SwitchStmt	0.000,0	0.000,9	−0.000,9

If we test the separate summations of artificial and realworld codebases then the null-hypothese is rejected for every criteria. (Table IX).

C. Comparisons between codebases

Appendices D-E gives an overview of the codebases and their respective test values over the success rate of aggregated statement recovery for the aggregated codebases group. For artificial and realworld code, we can observe the following:

TABLE VIII: Aggregated set: most stopped, most rejected, and effect size range.

Metric	Value
Most stopped (IDs)	1, 3, 4, 8 (3 stops each)
Most rejected (IDs)	5, 6, 7, 13, 14 (14 rejections each)
Effect size range (absolute)	0.0004 - 0.6795

1) The artificial code variant pairings 1-2 and 2-3 reject the null hypothese but 1-3, 1-4 and 3-4 did not.
2) Yarpgen, was rejected in every (artificial) pairing.
3) For the sqlite variants 10-12 and 11-12 were rejected but not 10-11, however note that the rejected effect size was ”only” about 1.1%.

Artificial code generators and their variants contrasted with the sqlite codebases in effect size, ranging from 0.0023 to 0.3247. An explanation for this is that the codegenerators are developed to test different parts of code, which explains their lack of similarity. Overall the results show that almost all pairings reject the null hypothesis, with 9 out of 105 being in the stop group, suggesting that Ghidra’s decompilation performance varies significantly between codebases.

TABLE IX: Artificial (ID 0) vs. realworld (ID 1) comparisons across statement groups.

Comparison 0-1	P-value	Holm Threshold	Decision	CI Estimate	CI Error
General	4.80×10^{-115}	5.00×10^{-2}	reject	2.24×10^{-2}	1.92×10^{-3}
CallExpr	1.04×10^{-149}	5.00×10^{-2}	reject	-3.89×10^{-2}	2.92×10^{-3}
ForStmt	1.11×10^{-16}	5.00×10^{-2}	reject	-2.09×10^{-2}	4.95×10^{-3}
IfStmt	2.25×10^{-222}	5.00×10^{-2}	reject	1.13×10^{-1}	6.94×10^{-3}
WhileStmt	5.67×10^{-370}	5.00×10^{-2}	reject	3.31×10^{-1}	1.58×10^{-2}

TABLE X: Delta proportion (artificial - real-world) most numerous recoveries per construct.
1. proportion = count of a specific markee (e.g., BinaryOperator) / total recoveredAsMarkee for that construct.
2. delta = artificial proportion - realworld Proportion.

Construct	**Self**		**1st alt**		**2nd alt**		**3rd alt**	
	Stmt	delta	Stmt	delta	Stmt	delta	Stmt	delta
CallExpr	CallExpr	−0.045,3	BinaryOperator	−0.078,8	ImplicitCast	0.135,3	IfStmt	−0.008,2
DeclStmt	DeclStmt	−0.000,5	BinaryOperator	−0.077,1	CallExpr	−0.001,4	IfStmt	−0.018,9
ForStmt	ForStmt	−0.020,9	BinaryOperator	−0.147,5	WhileStmt	−0.033,3	DoStmt	−0.015,8
GotoStmt	GotoStmt	−0.836,8	LabelStmt	−0.059,0	ReturnStmt	−0.048,4	BinaryOperator	−0.026,6
IfStmt	IfStmt	0.112,8	BinaryOperator	−0.154,8	ReturnStmt	−0.000,4	ImplicitCast	0.001,8
WhileStmt	WhileStmt	0.331,1	BinaryOperator	−0.131,6	ForStmt	−0.145,5	DoStmt	−0.081,5

1) Comparisons on the statement-level: For, If and While.: Appendices D, F, G and H, are required for this section.

The top scoring statements tend to evaluate to lower rejection rates compared to the aggregated set.

1) `CallExpr`: 82 of 86 pairings rejected
2) `ForStmt`: 38 of 55 pairings rejected
3) `IfStmt`: 79 of 102 pairings rejected
4) `WhileStmt`: 21 of 45 pairings rejected

For artifical code generators, CSmith paired with its variant and other codegenerators do not always reject the hypothesis for ”if” recovery. However the sqlite variants remain consistent within the different statement categories. Evidently decompilers exhibit different performance results for different statements and different characteristics.

A separate inspection using the correction restricted to the construct type itself shows no definitive similarity (appendix F, G, H, J):

The artificial codebases are the most rejected for `ForStmt` (Table XI), where DSmith does not generate any. In particular, results show no strict divide with ”artificial” - ”realworld” pairings being rejected categorically.

TABLE XI: Aggregated: most frequently rejected and stopped samples per statement type.

Statement	**Top 3 most rejected (ID, Count)**	**Most stopped (ID, Count)**
ForStmt	(1, 10), (2, 10), (14, 10)	(6, 6), (8, 6)
IfStmt	(7, 14), (1, 13), (6, 13)	(0, 7)
WhileStmt	(4, 8), (3, 8), (9, 5)	(13, 7)

V. Conclusion

Ghidra’s decompilation performance varied across codebase, statement class and codebase groups. Within the set of artificial code generators, we have observed rejections of the null hypothesis, likewise for the real-world set. Codebases can remain consistent within their variations, but performance does fluctuate. Each sample within different codebase-pairs rejected the hypothesis under circumstances, characteristics are unlikely to effect performance as much as the effect of particular statement classes. Within a class, codebases display different rejection results, suggesting that even within the statement class, specific statement instances have significant influence, contributing to the rejection of the null hypothesis if they are specific to a codebase. For the set of artificial and real-world codebases, the main factors relevant for testing are thus the diversity of statements, in particular within statement classes that each already have inherent decompilation challenges. Semantic roles are involved in the difficulty regarding the recovery of attribute details, declarations as well as the diversity of statements. For the evaluated Ghidra version there is a notable room for improvement regarding the restoration of case and switch statements, which are almost completely lost. Regarding decompiler testing, codegenerators that account for `SwitchStmt` and `CaseStmt` will complement the current set of tools, alongside investigating their alternative recovery constructs like `IfStmt` and `BinaryOperator`.

A. Limitations

The technique relies on compilers recognizing and correctly handling the added attributes, where each compilers can implement their own handling and formatting of the attribute specifications. In addition the recognition of markers, through names or otherwise is required, but heuristics could be developed to mitigate name loss.

Markers insertion is currently conservative, the presence of any macro causes the insertion to considered a risk and thus it aborts that insertion. This can cause statements to go unmarked that could have been marked.

The principles remain the same, create statements that ”anchor” surrounding code next to the markers. There is also the case as to why optimization is detrimental to this specific form of assessment heuristic, we reason the following:

1) Assume an equivalence relation where all optimization forms of some codebase are part of this set and where these forms optimize to some fully optimized form.

2) Assume we have some optimization function, which brings some form to a incremented form of optimization.

This gives a mapping of n-to-1 fully optimized form, for a complete optimization pass. Because the determination of the equivalence of such forms is undecidable, we cannot prove for any intermediate form to be representative of the original marked code. In addition, each layer of optimization can contribute to compilation bugs. Furthermore it would require the direct syntactic mapping mechanism to also alternatively evaluate for semantic equivalence between any arbitrary code. If we want evaluate decompilation with optimization through markers, then we need a more sophisticated heuristic. It would probably be better to start with evaluating singular optimizations from phase to phase, instead of compounded passes in single pass.

VI. Future work

A. Instrumentation

Macro handling is an area relatively untouched regarding source code instrumentation. Different approaches can be taken for macros concerning instrumentation:

- pre-expand macro's and source to macrofree code and then conduct instrumentation.
- improve heuristic for safe insertion around macro's.

The instrumentation in "Mage" is applied to a subset of possible constructs, there are other constructs to be included that complement this subset.

Mage uses the GNU attribute syntax to preserve markers that are understood by Clang and GCC, but not necessarily for other compilers. Starting from C23, the language specification has included attribute syntax inherent to C [35]. Decoupled attribute addition from the compiler implementation, would encourage the use of other compilers for measuring effects. An alternative is to integrate the compiler specific attribute implementation.

B. Evaluation heuristic

There are still certain categories of code that can be evaluated like libraries and headers, as currently the instrumentation focuses on codebase's local C source code only. However special care needs to be taken as these categories can be macro heavy.

For criteria's, additional information regarding readability scoring could be implemented using Campbell's metric[], cyclomatic complexity or Eom et al.'s [7] R2i.

For optimization, different heuristics could be developed for assessment that focuses on semantic aspects. A tweak that could be beneficial to heuristics that focus more deeply on semantics is to ensure "enclosement" around statements, i.e. making sure a statement has a marker before and after it. This at least provides an optimization barrier to the last statement in body ensuring there are no spillover statements under the marker. For example, a compiler can include the last statement of a body with possible control statement clauses after it from a different scope if left open.

For the recovery heuristic, a more lenient evaluation could be applied by inspecting for the presence of a statement within the bounds of two markers instead of only directly after it. This heuristic criteria would not evaluate the order of the markee result as strictly and puts more emphasis on the existence of a potential recovery. Do note that the recovery cases we examined with the Clang AST are particular to Ghidra, another decompiler might not extract operators, preassign variables or flatten control structures the same way.

C. Evaluation targets and applications

Currently this methodology is applied to the C language, other languages could also be interesting to investigate. The evaluations are used symmetrically within the language, maybe there are possibilities to use this mechanism to evaluate constructs between different languages.

Conditions such as different decompilers, both algorithmically and LLM's, can be assessed along with characteristics like operating system, architecture and etc.

The markers definition can also be modified to include sideeffects that can callback, opening the possibility of runtime analysis where the markers echo back information in certain points during execution.

Codesnippets can be extracted using (enclosed) markers if analysis requires the testing of specific sections of code, similar to existing function extraction methods. Training LLM's is a possible use case, similar to SALT [5] or alternatively one could investigate if isolated codesnippets decompile in a similar matter.

Not all statements are currently generated by the included group of artificial code generators. If there is no complementary code generator that is capable of generating missing constructs, then implementating capable generators is beneficial for decompiler testing. The summarizing functionality could also be expanded to search for sparse or missing occurences of specific constructs attributes, where the results can be used to assess code generators.

REFERENCES

[1] The angr community, "angr: A powerful and user-friendly binary analysis platform," https://angr.io/, 2026, accessed: 2026-01-30.

[2] National Security Agency, "Ghidra: Software Reverse Engineering Framework," https://ghidra-sre.org/, 2026, accessed: 2026-01-30.

[3] Hex-Rays, "IDA Pro Disassembler," https://hex-rays.com/ida-pro/, 2026, accessed: 2026-01-30.

[4] *Binary Ninja*, Vector 35 Inc., commercial interactive decompiler, disassembler, and binary analysis platform. [Online]. Available: https://binary.ninja/

[5] Y. Wang, X. Xu, X. Zhu, X. Gu, and B. Shen, "Salt4decompile: Inferring source-level abstract logic tree for llm-based binary decompilation," *arXiv preprint arXiv:2509.14646*, 2025.

[6] G. A. Campbell, "Cognitive complexity: an overview and evaluation," in *Proceedings of the 2018 International Conference on Technical Debt*, ser. TechDebt '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 57–58. [Online]. Available: https://doi.org/10.1145/3194164.3194186

[7] H. Eom, D. Kim, S. Lim, H. Koo, and S. Hwang, "R2i: A relative readability metric for decompiled code," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 383–405, 2024.

[8] *DWARF Debugging Information Format, Version 6*, DWARF Debugging Information Format Committee, 2025. [Online]. Available: https://snapshots.sourceware.org/dwarfstd/dwarf-spec/2025-05-05_22-29_1746484141/dwarf6-20250505-2228.pdf

[9] T. Theodoridis, M. Rigger, and Z. Su, "Finding missed optimizations through the lens of dead code elimination," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 697–709.

[10] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Boston, MA, USA: Addison-Wesley, 2006.

[11] C. Cifuentes, *Reverse compilation techniques*. Queensland University of Technology, Brisbane, 1994.

[12] K. v. G. Jaap van den Bos and K. Reijer, "Deb'm: A benchmarking platform for decompilers," 2024.

[13] H. Tan, X. Tian, H. Qi, J. Liu, Z. Gao, S. Wang, Q. Luo, J. Li, and Y. Zhang, "Decompile-bench: Million-scale binary-source function pairs for real-world binary decompilation," *arXiv preprint arXiv:2505.12668*, 2025.

[14] Z. Gao, Y. Cui, H. Wang, S. Qin, Y. Wang, Z. Bolun, and C. Zhang, "Decompilebench: A comprehensive benchmark for evaluating decompilers in real-world scenarios," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 250–23 267.

[15] C. Pang, T. Zhang, R. Yu, B. Mao, and J. Xu, "Ground truth for binary disassembly is not easy," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2479–2495.

[16] Y. Cao, R. Zhang, R. Liang, and K. Chen, "Evaluating the effectiveness of decompilers," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 491–502. [Online]. Available: https://doi.org/10.1145/3650212.3652144

[17] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2011, pp. 283–294.

[18] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for c and c++ compilers with yarpgen," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020. [Online]. Available: https://doi.org/10.1145/3428264

[19] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Transactions of the American Mathematical society*, vol. 74, no. 2, pp. 358–366, 1953.

[20] LLVM Developer Group, *Clang: A C-Language Front-End for LLVM*, Dec. 2025, accessed: July 2026. [Online]. Available: https://releases.llvm.org/21.1.0/docs/ReleaseNotes.html

[21] LLVM Project. (2026) clang: Class index. Generated via Doxygen. Accessed: 2026-07-20. [Online]. Available: https://clang.llvm.org/doxygen/classes.html

[22] Penn State Eberly College of Science, "Lesson 9.4: Comparing two proportions (the two-proportion z-test)," https://psu.edu, 2026, sTAT 415: Introduction to Mathematical Statistics. Covers hypothesis testing and confidence intervals using the two-proportion z-test.

[23] H.-Y. Kim, "Statistical notes for clinical researchers: Sample size calculation 2. comparison of two independent proportions," *Restorative Dentistry & Endodontics*, vol. 41, no. 2, pp. 154–158, May 2016, emphasizing sample size determination for the Fisher's exact test in small sample independent proportion comparisons.

[24] Kent State University Libraries, "SPSS tutorials: Chi-square test of independence," Kent State University LibGuides, July 2026, accessed: 2026-07-20. [Online]. Available: https://libguides.library.kent.edu/spss/chisquare

[25] Penn State Department of Statistics, "Lesson 27: Likelihood ratio tests," Penn State Statistics Online Courses, accessed: July 19, 2026. [Online]. Available: https://online.stat.psu.edu/stat415/book/export/html/847

[26] C. C. Serdar, M. Cihan, D. Yücel, and M. A. Serdar, "Sample size, power and effect size revisited: simplified and practical approaches in pre-clinical, clinical and laboratory studies," *Biochemia medica*, vol. 31, no. 1, pp. 27–53, 2021.

[27] M. Nath, "Multiple testing (FWER & FDR): Analysis of variance using R: Part 1," https://shiny.abdn.ac.uk/Stats/tutorials/R_ANOVA/R_Anova_P1/multiple-testing-fwer-fdr.html, 2026, university of Aberdeen, School of Medicine, Medical Sciences and Nutrition. Accessed: 2026-07-19.

[28] E. W. Weisstein, "Bonferroni correction," 2026, accessed: 2026-07-19. [Online]. Available: https://mathworld.wolfram.com/BonferroniCorrection.html

[29] SciPy Community, *scipy.stats.tukey_hsd — SciPy v1.10.1 Reference Guide*, 2023, accessed: 2026-07-19. [Online]. Available: https://eksamensvedlegg.it.ntnu.no/Felles/scipy.1101/reference/generated/scipy.stats.tukey_hsd.html

[30] D. Groppe, "Bonferroni-Holm Correction for Multiple Comparisons," https://nl.mathworks.com/matlabcentral/fileexchange/28303-bonferroni-holm-correction-for-multiple-comparisons, 2010, mATLAB Central File Exchange; accessed July 19, 2026.

[31] H. Seo, "Understanding dunnett's test," University of Virginia Library StatLab Articles, March 2026, accessed: 2026-07-19. [Online]. Available: https://library.virginia.edu/data/articles/understanding-dunnetts-test

[32] H. Abdi, "The Bonferroni and šidák corrections for multiple comparisons," in *Encyclopedia of Measurement and Statistics*, N. J. Salkind, Ed. Thousand Oaks, CA: Sage, 2007, pp. 103–107. [Online]. Available: https://www.utdallas.edu/~herve/Abdi-Bonferroni2007-pretty.pdf

[33] H. Guillermou, C. Abraham, I. Annesi-Maesano, and N. Molinari, "Multiple hypothesis testing in allergy and hypersensitivity diseases investigation: a pedagogical perspective," *Journal of Allergy and Hypersensitivity Diseases*, vol. 3, p. 100014, 2024.

[34] Y. Cao, R. Zhang, R. Liang, and K. Chen, "Evaluating the effectiveness of decompilers," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 491–502.

[35] cppreference.net contributors, "Attribute specifier sequence (C23): Attribute testing," cppreference.net, July 2026, accessed: 2026-07-22. [Online]. Available: https://cppreference.net/c/language/attributes.html#Attribute_testing

APPENDIX A
SUMMARY EXAMPLE (SHORTENED)


```
{
  "constructs": {
    "CallExpr": {
      "P(X)": 0.8786791125273228,
      "VAR(X)": 0.1066021297355192,
      "decomTotal": 3769973,
      "oracleTotal": 5186208,
      "recoveredAsMarkee": {
        "BinaryOperator": 222349,
        "BreakStmt": 163,
        "CallExpr": 2359138,
        "DeclStmt": 3743,
        "ForStmt": 633,
        "GotoStmt": 150,
        "IfStmt": 20887,
        "ImplicitCastExpr": 55862,
        "LabelStmt": 7,
        "ReturnStmt": 3336
      },
      "recoveryOfAttr": {
        "name_preserved": 1767634,
        "parity_arg_types": 1337027,
        "parity_args_count": 2292318,
        "parity_has_callee": 1769062,
        "parity_param_types": 731045,
        "parity_params_count": 1006797,
        "parity_side_effects": 2342786,
        "parity_variadic": 1688618,
        "recovered_construct": 2342794
      },
      "viableComparisons": 2666268
    },
    "CaseStmt": {
      "P(X)": 0.005679335954565312,
      "VAR(X)": 0.005647081097680494,
      "decomTotal": 14,
      "oracleTotal": 337758,
      "recoveredAsMarkee": {
        "BinaryOperator": 12,
        "CaseStmt": 13,
        "GotoStmt": 824,
        "LabelStmt": 1277,
        "ReturnStmt": 163
      },
      "recoveryOfAttr": {
        "recovered_construct": 13
      },
      "viableComparisons": 2289
    },
    "DeclStmt": {
      "P(X)": 0.0009415937258471401,
      "VAR(X)": 0.0009407071271025854,
      "decomTotal": 5183,
      "oracleTotal": 4938406,
      "recoveredAsMarkee": {
        "BinaryOperator": 1591082,
        "BreakStmt": 1,
        "CallExpr": 1363532,
        "DeclStmt": 1440,
        "DoStmt": 1089,
        "ForStmt": 12597,
        "GotoStmt": 171,
        "IfStmt": 77121,
        "ImplicitCastExpr": 3261,
        "LabelStmt": 109,
        "ReturnStmt": 6647,
        "WhileStmt": 1594
      },
      "recoveryOfAttr": {
        "parity_init_types": 0,
        "parity_single_decl": 2880,
        "recovered_alt_construct": 1172445,
        "recovered_construct": 2880
      },
      "viableComparisons": 3058644
    },
```

```
    "ForStmt": {
      "P(X)": 0.6268842884650163,
      "VAR(X)": 0.23390037734072652,
      "decomTotal": 279486,
      "oracleTotal": 666294,
      "recoveredAsMarkee": {
        "BinaryOperator": 101225,
        "BreakStmt": 4,
        "CallExpr": 14717,
        "DoStmt": 5441,
        "ForStmt": 235711,
        "GotoStmt": 3,
        "IfStmt": 987,
        "ImplicitCastExpr": 158,
        "LabelStmt": 1149,
        "ReturnStmt": 1910,
        "WhileStmt": 14699
      },
      "recoveryOfAttr": {
        "parity_cond_expr_types": 186895,
        "parity_cond_stmt_class": 168285,
        "parity_cond_var_stmt_class": 235711,
        "parity_cond_var_types": 235711,
        "parity_has_cond_expr": 235711,
        "parity_has_cond_var": 235711,
        "parity_has_incr": 234692,
        "parity_has_init": 230457,
        "parity_incr_expr_types": 216686,
        "parity_incr_stmt_class": 41533,
        "parity_init_stmt_class": 219572,
        "parity_init_types": 215429,
        "recovered_construct": 235711
      },
      "viableComparisons": 376004
    },
    "GotoStmt": {
      "P(X)": 0.8361558333115884,
      "VAR(X)": 0.1369992557305916,
      "decomTotal": 65417,
      "oracleTotal": 294320,
      "recoveredAsMarkee": {
        "BinaryOperator": 2056,
        "BreakStmt": 1735,
        "CallExpr": 166,
        "DoStmt": 163,
        "GotoStmt": 64088,
        "IfStmt": 168,
        "ImplicitCastExpr": 3,
        "LabelStmt": 4523,
        "ReturnStmt": 3741,
        "WhileStmt": 3
      },
      "recoveryOfAttr": {
        "recovered_construct": 64088
      },
      "viableComparisons": 76646
    },
    "IfStmt": {
      "P(X)": 0.7174446047671779,
      "VAR(X)": 0.20271784385764582,
      "decomTotal": 2757030,
      "oracleTotal": 5886251,
      "recoveredAsMarkee": {
        "BinaryOperator": 1003105,
        "BreakStmt": 163,
        "CallExpr": 2917,
        "ForStmt": 6,
        "GotoStmt": 173,
        "IfStmt": 2565668,
        "ImplicitCastExpr": 894,
        "LabelStmt": 6,
        "RecoveryExpr": 326,
        "ReturnStmt": 2715,
        "UnaryOperator": 144,
        "WhileStmt": 3
      },
      "recoveryOfAttr": {
        "parity_cond_expr_types": 1941635,
        "parity_cond_stmt_class": 1210840,
```

```
        "parity_cond_var_stmt_class": 2565668,
        "parity_cond_var_types": 2565668,
        "parity_has_cond_expr": 2565668,
        "parity_has_cond_var": 2565668,
        "parity_has_else": 2142800,
        "recovered_construct": 2565668
      },
      "viableComparisons": 3576120
    },
    "SwitchStmt": {
      "P(X)": 0.0009116976692890229,
      "VAR(X)": 0.0009108664766488358,
      "decomTotal": 62,
      "oracleTotal": 125008,
      "recoveredAsMarkee": {
        "BinaryOperator": 32962,
        "IfStmt": 34760,
        "ReturnStmt": 221,
        "SwitchStmt": 62
      },
      "recoveryOfAttr": {
        "parity_cond_expr_types": 40,
        "parity_cond_stmt_class": 61,
        "parity_cond_var_stmt_class": 62,
        "parity_cond_var_types": 62,
        "parity_has_cond_expr": 62,
        "parity_has_cond_var": 62,
        "parity_has_init": 62,
        "parity_has_switch_cases": 0,
        "parity_init_stmt_class": 62,
        "parity_init_types": 62,
        "parity_switch_cases": 0,
        "recovered_construct": 62
      },
      "viableComparisons": 68005
    },
    "WhileStmt": {
      "P(X)": 0.471287938668008,
      "VAR(X)": 0.24917561753406792,
      "decomTotal": 101565,
      "oracleTotal": 328422,
      "recoveredAsMarkee": {
        "BinaryOperator": 38951,
        "CallExpr": 340,
        "DoStmt": 20694,
        "ForStmt": 30338,
        "GotoStmt": 4,
        "IfStmt": 187,
        "LabelStmt": 4108,
        "ReturnStmt": 1031,
        "WhileStmt": 85264
      },
      "recoveryOfAttr": {
        "parity_cond_expr_types": 42424,
        "parity_cond_stmt_class": 18725,
        "parity_cond_var_stmt_class": 85264,
        "parity_cond_var_types": 85264,
        "parity_has_cond_expr": 85264,
        "parity_has_cond_var": 85264,
        "recovered_construct": 85264
      },
      "viableComparisons": 180917
    }
  },
  "metadata": {
    "P(X)": 0.5234381131246622,
    "VAR(X)": 0.24945065485315554,
    "decomTotal": 10129975,
    "deterioratedConstructsTotal": 4827560,
    "findingsTotal": 18697540,
    "markerDuplication": 1810,
    "oracleMissingTotal": 50101,
    "oracleTotal": 18647439,
    "recoveredConstructsTotal": 5302415,
    "viableComparisons": 10129975
  }
}
```

APPENDIX B
CONSTRUCT CRITERIA

Construct criteria follow the general outline:

1) Construct class recovery.
2) Construct attribute recovery.

Note that the current set of evaluable attributes is not exhaustive and based on builtin available Clang methods to extract direct properties. The is recorded for each construct:

1) CaseStmt, a case statement within a switch. Is only evaluated on construct presence.
2) CallExpr, represents a function call.
   a) has_callee - the callee of a callexpr is the function belonging to the callexpr that gets invoked. If the callee can be detected, then we can extract extra properties derived from the markee:
      i) is_variadic - checks if the function can hold any number of arguments.

```
void print(int n, ...) // example variadic function
```

      ii) num_params - records the amount of explicitly declared parameters..
      iii) params - records the type and checks if these are either pointer or const of the explicitly declared parameters.
   b) return_type - the type of the call expression's returned value.
   c) has_side_effects - check if a function has sideeffects, we use Clang's innate methods for this.
   d) num_args - counts the number of provided arguments to the callexpression.
   e) arg_types - checks for each arg; the type and if it is a pointer and/or constant.
3) DeclStmt, a class that includes declarations with statements and expressions.
   a) single_decl - checks if the markee is a single declaration based on the number of variable declarations.

```
int i=0. // single_decl = true
int i=0, j=1. // multiple declarations, single_decl = false
```

   b) has_=_init - checks if the initialization uses the "=" operator.
4) ForStmt, represents a for-loop statement.
   a) Init clause:
      i) has_init - checks the presence of the clause.
      ii) init_stmt_class - if the clause is there then checks for its statement class.
      iii) init_types - records the type and checks if the it is a pointer or const.
   b) Condition clause:
      i) has_cond - checks the presence of the clause.
      ii) cond_stmt_class - if the clause is there then checks for its statement class.
      iii) cond_types - records the type and checks if the it is a pointer or const.
   c) Increment clause:
      i) has_incr - checks the presence of the clause.
      ii) incr_stmt_class - if the clause is there then checks for its statement class.
      iii) incr_types - records the type and checks if the it is a pointer or const.
5) GotoStmt, a statement that transfers control to a label. Is only evaluated on construct presence.
6) IfStmt, represent an if statement.
   a) Condition clause:
      i) has_cond - checks the presence of the clause.
      ii) cond_stmt_class - if the clause is there then checks for its statement class.
      iii) cond_types - records the type and checks if the it is a pointer or const.
   b) parity_has_else checks for the existence of at least one or more else/else if extension.
7) SwitchStmt, represents an switch statement.
   - Condition clause:
     – has_cond - checks the presence of the clause.
     – cond_stmt_class - if the clause is there then checks for its statement class.
     – cond_types - records the type and checks if the it is a pointer or const.
8) WhileStmt - represents a while-loop statement.
   - Condition clause:
     – has_cond - checks the presence of the clause.
     – cond_stmt_class - if the clause is there then checks for its statement class.
     – cond_types - records the type and checks if the it is a pointer or const.

APPENDIX C
BENCHMARK RESULTS WITH GHIDRA_12.1.2_PUBLIC_20260605

A. Realworld

TABLE XII: Benchmark(any, real-world, algorithmic, ghidra), clang construct and mapping recovery metrics.

Construct Type	P(X)	VAR(X)	Viable Comp.	Recovered	Oracle Total	Decom. Total
CallExpr	0.8808	0.1050	2,533,009	2,231,037	4,976,406	3,594,311
CaseStmt	0.0058	0.0058	2,240	13	334,680	13
DeclStmt	0.0010	0.0010	2,956,450	2,880	4,759,338	5,183
ForStmt	0.6294	0.2332	328,954	207,054	561,958	244,118
GoToStmt	0.8368	0.1365	75,731	63,374	291,939	64,656
IfStmt	0.7159	0.2034	3,530,785	2,527,619	5,808,407	2,716,775
SwitchStmt	0.0009	0.0009	67,631	60	124,604	60
WhileStmt	0.4640	0.2487	175,785	81,566	322,647	97,334

TABLE XIII: Benchmark(any, real-world, algorithmic, ghidra), markee recovery metrics (1 of 2).

	Recovered as markee						
Construct	BinaryOperator	BreakStmt	CallExpr	CaseStmt	DeclStmt	DoStmt	ForStmt
CallExpr	220,248	163	2,247,173	0	3,743	0	615
CaseStmt	4	0	0	13	0	0	0
DeclStmt	1,544,919	0	1,317,025	0	1,440	996	6,329
ForStmt	94,106	0	4,681	0	0	5,382	207,054
GoToStmt	2,017	1,725	156	0	0	163	0
IfStmt	996,853	163	2,043	0	0	0	1
SwitchStmt	32,790	0	0	0	0	0	0
WhileStmt	38,448	0	145	0	0	20,433	29,930

TABLE XIV: Benchmark(any, real-world, algorithmic, ghidra), markee recovery metrics (2 of 2).

	Recovered as markee							
Construct	GoToStmt	IfStmt	LabelStmt	RecoverExpr	ReturnStmt	SwitchStmt	UnaryOperator	WhileStmt
CallExpr	146	20,691	0	0	3,317	0	0	0
CaseStmt	808	0	1,252	0	163	0	0	0
DeclStmt	163	76,095	1	0	5,469	0	0	1,564
ForStmt	0	491	1,134	0	1,760	0	0	14,202
GoToStmt	63,374	166	4,467	0	3,662	0	0	0
IfStmt	164	2,527,619	1	326	2,697	0	144	1
SwitchStmt	0	34,560	0	0	221	60	0	0
WhileStmt	1	146	4,085	0	1,031	0	0	81,566

B. Artificial

TABLE XV: Benchmark(any, artificial, algorithmic, ghidra), clang construct and mapping recovery metrics.

Construct Type	P(X)	VAR(X)	Viable Comp.	Recovered	Oracle Total	Decom. Total
CallExpr	0.8419	0.1331	116,530	98,107	191,006	144,918
CaseStmt	0	0	0	0	0	0
DeclStmt	0.00	0.00	80,530	0	155,429	0
ForStmt	0.6085	0.2382	44,137	26,857	101,223	33,205
GoToStmt	0.00	0.00	0	0	8	13
IfStmt	0.8287	0.1420	19,862	16,459	49,711	17,532
SwitchStmt	0	0	0	0	0	0
WhileStmt	0.7951	0.1629	3,914	3,112	4,491	3,571

TABLE XVI: Benchmark(any, artificial, algorithmic, ghidra), markee recovery metrics (1 of 2).

	Recovered as markee						
Construct	BinaryOperator	BreakStmt	CallExpr	CaseStmt	DeclStmt	DoStmt	ForStmt
CallExpr	951	0	98,107	0	0	0	0
CaseStmt	0	0	0	0	0	0	0
DeclStmt	35,868	1	35,765	0	0	86	6,251
ForStmt	6,117	2	10,032	0	0	27	26,857
GoToStmt	0	0	0	0	0	0	0
IfStmt	2,533	0	819	0	0	0	0
SwitchStmt	0	0	0	0	0	0	0
WhileStmt	341	0	195	0	0	136	97

TABLE XVII: Benchmark(any, artificial, algorithmic, ghidra), markee recovery metrics (2 of 2).

	Recovered as markee							
Construct	GoToStmt	IfStmt	LabelStmt	RecoverExpr	ReturnStmt	SwitchStmt	UnaryOperator	WhileStmt
CallExpr	0	0	0	0	0	0	0	0
CaseStmt	0	0	0	0	0	0	0	0
DeclStmt	8	546	108	0	1,141	0	0	23
ForStmt	1	494	7	0	150	0	0	436
GoToStmt	0	0	0	0	0	0	0	0
IfStmt	4	16,459	1	0	7	0	0	0
SwitchStmt	0	0	0	0	0	0	0	0
WhileStmt	0	33	0	0	0	0	0	3,112

APPENDIX D
CODEBASE MAPPING AND CHARACTERISTICS.

TABLE XVIII: Real-world: mapping and characteristics of codebases.

ID	Codebase	Viable comp.	Arch	Bits	Binary	Class	Compiler	Endian	Lang	Machine	OS
0	AvdanOSImager	203	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
1	git-2.54.0	9,722,084	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
2	hashcat-7.1.2[1]	6,104	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c++	AMD x86-64	linux
3	radare2-6.1.8	5,099	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
4	redis-8.8.0	4,368	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
5	sqlite-amalgamation-3530300	34,236	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
6	tmux-3.6b	14,449	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux

[1]C files instrumented only.

TABLE XIX: Artificial: mapping and characteristics of codebases.

ID	Codebase	Viable comp.	Arch	Bits	Binary	Class	Compiler	Endian	Lang	Machine	OS
0	csmith	61,216	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
1	csmith	90,444	x86	32	pe	PE32	LLVM MinGW[2]	little	c	i386	windows
2	dsmith[3]	50,092	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
3	dsmith[3]	33,597	x86	32	pe	PE32	LLVM MinGW[2]	little	c	i386	windows
4	yarpgen-codegen	29,624	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux

[2]Compiler: GCC (15.1.1 20250521 (Fedora MinGW 15.1.1-1.fc43)).

[3]Modification made to DSmith's source.py to allow seeds.

TABLE XX: Aggregated: mapping and characteristics of codebases used.

ID	Codebase	Viable comp.	Arch	Bits	Binary	Class	Compiler	Endian	Lang	Machine	OS
0	AvdanOSImager	203	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
1	csmith	61,216	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
2	csmith	90,444	x86	32	pe	PE32	LLVM MinGW[2]	little	c	i386	windows
3	dsmith[3]	50,092	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
4	dsmith[3]	33,597	x86	32	pe	PE32	LLVM MinGW[2]	little	c	i386	windows
5	git-2.54.0	9,722,084	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
6	hashcat-7.1.2[1]	6,104	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c++	AMD x86-64	linux
7	radare2-6.1.8	5,099	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
8	redis-8.8.0	4,368	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
9	rufus	11,389	x86	64	pe	PE32+	MinGW[2]	little	c	AMD 64	windows
10	sqlite-amalgamation-3530300	34,236	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
11	sqlite-amalgamation-3530300	33,517	x86	32	pe	PE32	MinGW[2]	little	c	i386	windows
12	sqlite-amalgamation-3530300	33,553	x86	64	pe	PE32+	MinGW[2]	little	c	AMD 64	windows
13	tmux-3.6b	14,449	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux
14	yarpgen-codegen	29,624	x86	64	elf	ELF64	GCC (GNU) 15.2.1 20260123 (Red Hat 15.2.1-7)	little	c	AMD x86-64	linux

[1] C files instrumented only.

[2] Compiler: GCC (15.1.1 20250521 (Fedora MinGW 15.1.1-1.fc43)).

[3] Modification made to DSmith's source.py to allow seeds.

APPENDIX E
AGGREGATED STATEMENT STATISTICS

A. Holm-Bonferroni tables: aggregated general

TABLE XXI: Holm-Bonferroni correction for statement types combined

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
2-5	0	4.76×10^{-4}	reject	2.21×10^{-1}	2.86×10^{-3}
2-7	0	4.81×10^{-4}	reject	6.79×10^{-1}	7.33×10^{-3}
5-7	$3.15 \times 10^{-3,845}$	4.85×10^{-4}	reject	4.59×10^{-1}	6.76×10^{-3}
1-2	$4.79 \times 10^{-3,666}$	4.90×10^{-4}	reject	-3.21×10^{-1}	4.84×10^{-3}
2-3	$2.80 \times 10^{-3,289}$	4.95×10^{-4}	reject	3.25×10^{-1}	5.17×10^{-3}
7-14	$6.02 \times 10^{-2,416}$	5.00×10^{-4}	reject	-4.74×10^{-1}	8.82×10^{-3}
2-4	$3.26 \times 10^{-2,348}$	5.05×10^{-4}	reject	3.18×10^{-1}	6.00×10^{-3}
7-10	$1.44 \times 10^{-2,118}$	5.10×10^{-4}	reject	-4.32×10^{-1}	8.58×10^{-3}
7-11	$1.48 \times 10^{-2,105}$	5.16×10^{-4}	reject	-4.33×10^{-1}	8.62×10^{-3}
7-12	$3.61 \times 10^{-1,999}$	5.21×10^{-4}	reject	-4.21×10^{-1}	8.61×10^{-3}
7-13	$1.79 \times 10^{-1,847}$	5.26×10^{-4}	reject	-4.96×10^{-1}	1.05×10^{-2}
1-7	$3.16 \times 10^{-1,766}$	5.32×10^{-4}	reject	3.59×10^{-1}	7.81×10^{-3}
3-7	$2.50 \times 10^{-1,636}$	5.38×10^{-4}	reject	3.55×10^{-1}	8.02×10^{-3}
2-12	$3.18 \times 10^{-1,517}$	5.44×10^{-4}	reject	2.58×10^{-1}	6.06×10^{-3}
4-7	$7.51 \times 10^{-1,482}$	5.50×10^{-4}	reject	3.61×10^{-1}	8.58×10^{-3}
2-10	$3.41 \times 10^{-1,414}$	5.56×10^{-4}	reject	2.47×10^{-1}	6.01×10^{-3}
2-11	$2.83 \times 10^{-1,386}$	5.62×10^{-4}	reject	2.47×10^{-1}	6.06×10^{-3}
7-9	$5.44 \times 10^{-1,066}$	5.68×10^{-4}	reject	-4.07×10^{-1}	1.14×10^{-2}
2-14	1.60×10^{-872}	5.75×10^{-4}	reject	2.05×10^{-1}	6.35×10^{-3}
2-9	3.51×10^{-677}	5.81×10^{-4}	reject	2.73×10^{-1}	9.60×10^{-3}
2-6	1.93×10^{-675}	5.88×10^{-4}	reject	3.56×10^{-1}	1.25×10^{-2}
1-5	1.13×10^{-542}	5.95×10^{-4}	reject	-9.99×10^{-2}	3.93×10^{-3}
3-5	3.12×10^{-483}	6.02×10^{-4}	reject	-1.04×10^{-1}	4.33×10^{-3}
6-7	6.43×10^{-449}	6.10×10^{-4}	reject	3.23×10^{-1}	1.40×10^{-2}
2-8	3.11×10^{-405}	6.17×10^{-4}	reject	3.27×10^{-1}	1.49×10^{-2}
7-8	2.59×10^{-401}	6.25×10^{-4}	reject	-3.52×10^{-1}	1.61×10^{-2}
2-13	1.63×10^{-385}	6.33×10^{-4}	reject	1.84×10^{-1}	8.58×10^{-3}
4-5	6.37×10^{-286}	6.41×10^{-4}	reject	-9.77×10^{-2}	5.30×10^{-3}
3-14	4.10×10^{-237}	6.49×10^{-4}	reject	-1.20×10^{-1}	7.13×10^{-3}
1-14	8.69×10^{-237}	6.58×10^{-4}	reject	-1.16×10^{-1}	6.90×10^{-3}
3-13	5.51×10^{-199}	6.67×10^{-4}	reject	-1.41×10^{-1}	9.17×10^{-3}
1-13	2.03×10^{-195}	6.76×10^{-4}	reject	-1.37×10^{-1}	8.99×10^{-3}
4-14	2.73×10^{-180}	6.85×10^{-4}	reject	-1.13×10^{-1}	7.76×10^{-3}
4-13	9.11×10^{-164}	6.94×10^{-4}	reject	-1.35×10^{-1}	9.67×10^{-3}
6-13	7.86×10^{-118}	7.04×10^{-4}	reject	-1.73×10^{-1}	1.47×10^{-2}
0-7	1.95×10^{-110}	7.14×10^{-4}	reject	6.84×10^{-1}	6.00×10^{-2}
3-10	3.44×10^{-109}	7.25×10^{-4}	reject	-7.74×10^{-2}	6.84×10^{-3}
3-11	5.38×10^{-109}	7.35×10^{-4}	reject	-7.79×10^{-2}	6.88×10^{-3}
6-14	1.85×10^{-107}	7.46×10^{-4}	reject	-1.51×10^{-1}	1.35×10^{-2}
1-10	1.35×10^{-105}	7.58×10^{-4}	reject	-7.33×10^{-2}	6.59×10^{-3}
1-11	2.27×10^{-105}	7.69×10^{-4}	reject	-7.38×10^{-2}	6.63×10^{-3}
5-6	5.63×10^{-105}	7.81×10^{-4}	reject	1.36×10^{-1}	1.22×10^{-2}
3-12	2.07×10^{-80}	7.94×10^{-4}	reject	-6.66×10^{-2}	6.88×10^{-3}
4-11	2.11×10^{-77}	8.07×10^{-4}	reject	-7.15×10^{-2}	7.52×10^{-3}
4-10	2.63×10^{-77}	8.20×10^{-4}	reject	-7.11×10^{-2}	7.48×10^{-3}
1-12	2.40×10^{-76}	8.33×10^{-4}	reject	-6.25×10^{-2}	6.63×10^{-3}
8-13	1.51×10^{-63}	8.48×10^{-4}	reject	-1.43×10^{-1}	1.67×10^{-2}
6-11	2.75×10^{-58}	8.62×10^{-4}	reject	-1.10×10^{-1}	1.33×10^{-2}
6-10	4.88×10^{-58}	8.77×10^{-4}	reject	-1.09×10^{-1}	1.33×10^{-2}
4-12	1.43×10^{-55}	8.93×10^{-4}	reject	-6.03×10^{-2}	7.52×10^{-3}
8-14	1.01×10^{-52}	9.09×10^{-4}	reject	-1.22×10^{-1}	1.57×10^{-2}
12-13	6.92×10^{-51}	9.26×10^{-4}	reject	-7.43×10^{-2}	9.70×10^{-3}
6-12	2.60×10^{-47}	9.43×10^{-4}	reject	-9.83×10^{-2}	1.33×10^{-2}
5-8	2.53×10^{-46}	9.62×10^{-4}	reject	1.07×10^{-1}	1.46×10^{-2}
9-13	2.64×10^{-46}	9.80×10^{-4}	reject	-8.91×10^{-2}	1.22×10^{-2}
5-12	1.29×10^{-42}	1.00×10^{-3}	reject	3.74×10^{-2}	5.36×10^{-3}
12-14	1.43×10^{-40}	1.02×10^{-3}	reject	-5.31×10^{-2}	7.80×10^{-3}
10-13	7.74×10^{-38}	1.04×10^{-3}	reject	-6.35×10^{-2}	9.67×10^{-3}
11-13	3.99×10^{-37}	1.06×10^{-3}	reject	-6.30×10^{-2}	9.70×10^{-3}
9-14	4.82×10^{-35}	1.09×10^{-3}	reject	-6.79×10^{-2}	1.08×10^{-2}
0-6	3.37×10^{-31}	1.11×10^{-3}	reject	3.61×10^{-1}	6.09×10^{-2}
5-9	5.68×10^{-29}	1.14×10^{-3}	reject	5.23×10^{-2}	9.17×10^{-3}
0-3	3.92×10^{-27}	1.16×10^{-3}	reject	3.29×10^{-1}	5.98×10^{-2}

Continued on next page

TABLE XXI – continued from previous page

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
6-9	9.47×10^{-27}	1.19×10^{-3}	reject	-8.35×10^{-2}	1.53×10^{-2}
10-14	1.43×10^{-26}	1.22×10^{-3}	reject	-4.23×10^{-2}	7.76×10^{-3}
0-1	1.58×10^{-26}	1.25×10^{-3}	reject	3.25×10^{-1}	5.98×10^{-2}
0-8	3.36×10^{-26}	1.28×10^{-3}	reject	3.32×10^{-1}	6.14×10^{-2}
0-4	4.29×10^{-26}	1.32×10^{-3}	reject	3.23×10^{-1}	5.99×10^{-2}
11-14	7.88×10^{-26}	1.35×10^{-3}	reject	-4.18×10^{-2}	7.80×10^{-3}
8-11	4.13×10^{-24}	1.39×10^{-3}	reject	-8.05×10^{-2}	1.56×10^{-2}
8-10	6.28×10^{-24}	1.43×10^{-3}	reject	-8.00×10^{-2}	1.56×10^{-2}
3-9	1.45×10^{-23}	1.47×10^{-3}	reject	-5.17×10^{-2}	1.01×10^{-2}
5-10	8.92×10^{-23}	1.52×10^{-3}	reject	2.66×10^{-2}	5.31×10^{-3}
5-11	1.13×10^{-21}	1.56×10^{-3}	reject	2.62×10^{-2}	5.36×10^{-3}
1-9	7.39×10^{-21}	1.61×10^{-3}	reject	-4.76×10^{-2}	9.97×10^{-3}
0-9	2.04×10^{-19}	1.67×10^{-3}	reject	2.78×10^{-1}	6.04×10^{-2}
5-13	4.66×10^{-19}	1.72×10^{-3}	reject	-3.69×10^{-2}	8.10×10^{-3}
8-12	2.90×10^{-18}	1.79×10^{-3}	reject	-6.92×10^{-2}	1.56×10^{-2}
0-12	8.45×10^{-18}	1.85×10^{-3}	reject	2.63×10^{-1}	5.99×10^{-2}
4-9	4.35×10^{-17}	1.92×10^{-3}	reject	-4.54×10^{-2}	1.06×10^{-2}
0-10	1.72×10^{-16}	2.00×10^{-3}	reject	2.52×10^{-1}	5.99×10^{-2}
0-11	1.94×10^{-16}	2.08×10^{-3}	reject	2.51×10^{-1}	5.99×10^{-2}
0-5	1.37×10^{-13}	2.17×10^{-3}	reject	2.25×10^{-1}	5.97×10^{-2}
0-14	7.20×10^{-12}	2.27×10^{-3}	reject	2.10×10^{-1}	5.99×10^{-2}
8-9	6.73×10^{-10}	2.38×10^{-3}	reject	-5.43×10^{-2}	1.73×10^{-2}
0-13	8.66×10^{-10}	2.50×10^{-3}	reject	1.88×10^{-1}	6.02×10^{-2}
4-6	2.07×10^{-8}	2.63×10^{-3}	reject	3.81×10^{-2}	1.33×10^{-2}
1-6	4.50×10^{-8}	2.78×10^{-3}	reject	3.58×10^{-2}	1.28×10^{-2}
5-14	6.62×10^{-8}	2.94×10^{-3}	reject	-1.57×10^{-2}	5.68×10^{-3}
9-11	1.42×10^{-6}	3.13×10^{-3}	reject	-2.61×10^{-2}	1.06×10^{-2}
3-6	1.61×10^{-6}	3.33×10^{-3}	reject	3.17×10^{-2}	1.30×10^{-2}
9-10	1.98×10^{-6}	3.57×10^{-3}	reject	-2.57×10^{-2}	1.06×10^{-2}
13-14	2.64×10^{-5}	3.85×10^{-3}	reject	2.12×10^{-2}	9.89×10^{-3}
6-8	2.75×10^{-3}	4.17×10^{-3}	reject	-2.91×10^{-2}	1.91×10^{-2}
11-12	3.62×10^{-3}	4.55×10^{-3}	reject	1.12×10^{-2}	7.57×10^{-3}
10-12	4.89×10^{-3}	5.00×10^{-3}	reject	1.08×10^{-2}	7.53×10^{-3}
9-12	5.99×10^{-3}	5.56×10^{-3}	stop	-1.49×10^{-2}	1.06×10^{-2}
3-4	6.78×10^{-2}	6.25×10^{-3}	stop	-6.36×10^{-3}	6.83×10^{-3}
1-3	1.69×10^{-1}	7.14×10^{-3}	stop	4.09×10^{-3}	5.83×10^{-3}
4-8	2.58×10^{-1}	8.33×10^{-3}	stop	8.98×10^{-3}	1.55×10^{-2}
1-8	3.85×10^{-1}	1.00×10^{-2}	stop	6.70×10^{-3}	1.51×10^{-2}
1-4	4.98×10^{-1}	1.25×10^{-2}	stop	-2.27×10^{-3}	6.58×10^{-3}
3-8	7.37×10^{-1}	1.67×10^{-2}	stop	2.61×10^{-3}	1.52×10^{-2}
0-2	8.81×10^{-1}	2.50×10^{-2}	stop	4.56×10^{-3}	5.97×10^{-2}
10-11	9.12×10^{-1}	5.00×10^{-2}	stop	-4.26×10^{-4}	7.53×10^{-3}

APPENDIX F
FOR STATEMENT STATISTICS

A. Holm-Bonferroni tables: aggregated `ForStmt`

TABLE XXII: Holm-Bonferroni correction for ForStmt

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
1-2	$2.15 \times 10^{-3,013}$	9.09×10^{-4}	reject	-5.59×10^{-1}	9.31×10^{-3}
1-5	$3.24 \times 10^{-1,566}$	9.26×10^{-4}	reject	-3.33×10^{-1}	7.68×10^{-3}
2-5	$1.44 \times 10^{-1,281}$	9.43×10^{-4}	reject	2.27×10^{-1}	5.80×10^{-3}
2-14	1.18×10^{-551}	9.62×10^{-4}	reject	4.44×10^{-1}	1.73×10^{-2}
5-14	6.71×10^{-147}	9.80×10^{-4}	reject	2.17×10^{-1}	1.65×10^{-2}
1-11	2.96×10^{-122}	1.00×10^{-3}	reject	-3.51×10^{-1}	2.93×10^{-2}
1-12	3.17×10^{-119}	1.02×10^{-3}	reject	-3.47×10^{-1}	2.93×10^{-2}
1-10	1.92×10^{-118}	1.04×10^{-3}	reject	-3.44×10^{-1}	2.91×10^{-2}
1-13	4.04×10^{-67}	1.06×10^{-3}	reject	-4.59×10^{-1}	5.20×10^{-2}
2-9	1.05×10^{-61}	1.09×10^{-3}	reject	3.68×10^{-1}	4.35×10^{-2}
2-10	3.43×10^{-49}	1.11×10^{-3}	reject	2.16×10^{-1}	2.87×10^{-2}
2-12	1.55×10^{-47}	1.14×10^{-3}	reject	2.13×10^{-1}	2.88×10^{-2}
2-11	1.82×10^{-45}	1.16×10^{-3}	reject	2.08×10^{-1}	2.88×10^{-2}
11-14	2.55×10^{-45}	1.19×10^{-3}	reject	2.36×10^{-1}	3.27×10^{-2}
12-14	1.20×10^{-43}	1.22×10^{-3}	reject	2.31×10^{-1}	3.27×10^{-2}
10-14	5.90×10^{-43}	1.25×10^{-3}	reject	2.28×10^{-1}	3.26×10^{-2}
1-14	3.77×10^{-36}	1.28×10^{-3}	reject	-1.15×10^{-1}	1.80×10^{-2}
13-14	9.75×10^{-36}	1.32×10^{-3}	reject	3.44×10^{-1}	5.40×10^{-2}
1-6	1.46×10^{-18}	1.35×10^{-3}	reject	-3.27×10^{-1}	7.30×10^{-2}
1-9	1.04×10^{-17}	1.39×10^{-3}	reject	-1.91×10^{-1}	4.38×10^{-2}
9-13	5.36×10^{-15}	1.43×10^{-3}	reject	-2.68×10^{-1}	6.72×10^{-2}
2-8	4.94×10^{-11}	1.47×10^{-3}	reject	2.84×10^{-1}	8.46×10^{-2}
5-9	1.55×10^{-10}	1.52×10^{-3}	reject	1.41×10^{-1}	4.32×10^{-2}
1-8	1.83×10^{-10}	1.56×10^{-3}	reject	-2.76×10^{-1}	8.48×10^{-2}
2-6	4.26×10^{-10}	1.61×10^{-3}	reject	2.32×10^{-1}	7.28×10^{-2}
9-11	1.31×10^{-9}	1.67×10^{-3}	reject	-1.60×10^{-1}	5.16×10^{-2}
9-12	3.90×10^{-9}	1.72×10^{-3}	reject	-1.55×10^{-1}	5.16×10^{-2}
9-10	6.96×10^{-9}	1.79×10^{-3}	reject	-1.52×10^{-1}	5.15×10^{-2}
6-14	2.37×10^{-8}	1.85×10^{-3}	reject	2.12×10^{-1}	7.44×10^{-2}
5-13	1.37×10^{-6}	1.92×10^{-3}	reject	-1.27×10^{-1}	5.15×10^{-2}
10-13	1.11×10^{-4}	2.00×10^{-3}	reject	-1.16×10^{-1}	5.87×10^{-2}
2-13	1.55×10^{-4}	2.08×10^{-3}	reject	1.0×10^{-1}	5.18×10^{-2}
12-13	1.64×10^{-4}	2.17×10^{-3}	reject	-1.13×10^{-1}	5.87×10^{-2}
8-14	2.61×10^{-4}	2.27×10^{-3}	reject	1.60×10^{-1}	8.60×10^{-2}
8-13	2.70×10^{-4}	2.38×10^{-3}	reject	-1.84×10^{-1}	9.89×10^{-2}
11-13	3.04×10^{-4}	2.50×10^{-3}	reject	-1.08×10^{-1}	5.88×10^{-2}
9-14	1.24×10^{-3}	2.63×10^{-3}	reject	7.60×10^{-2}	4.62×10^{-2}
6-9	1.61×10^{-3}	2.78×10^{-3}	reject	1.36×10^{-1}	8.45×10^{-2}
6-13	3.65×10^{-3}	2.94×10^{-3}	stop	-1.32×10^{-1}	8.90×10^{-2}
8-9	8.18×10^{-2}	3.13×10^{-3}	stop	8.42×10^{-2}	9.48×10^{-2}
8-11	9.65×10^{-2}	3.33×10^{-3}	stop	-7.55×10^{-2}	8.90×10^{-2}
8-12	1.19×10^{-1}	3.57×10^{-3}	stop	-7.08×10^{-2}	8.90×10^{-2}
8-10	1.34×10^{-1}	3.85×10^{-3}	stop	-6.80×10^{-2}	8.90×10^{-2}
5-8	1.87×10^{-1}	4.17×10^{-3}	stop	5.68×10^{-2}	8.44×10^{-2}
5-11	1.97×10^{-1}	4.55×10^{-3}	stop	-1.87×10^{-2}	2.84×10^{-2}
5-12	3.34×10^{-1}	5.00×10^{-3}	stop	-1.40×10^{-2}	2.83×10^{-2}
6-8	3.62×10^{-1}	5.56×10^{-3}	stop	5.18×10^{-2}	1.11×10^{-1}
5-10	4.36×10^{-1}	6.25×10^{-3}	stop	-1.12×10^{-2}	2.82×10^{-2}
6-11	5.50×10^{-1}	7.14×10^{-3}	stop	-2.38×10^{-2}	7.79×10^{-2}
6-12	6.32×10^{-1}	8.33×10^{-3}	stop	-1.91×10^{-2}	7.79×10^{-2}
6-10	6.82×10^{-1}	1.00×10^{-2}	stop	-1.63×10^{-2}	7.79×10^{-2}
10-11	7.13×10^{-1}	1.25×10^{-2}	stop	-7.48×10^{-3}	3.99×10^{-2}
11-12	8.18×10^{-1}	1.67×10^{-2}	stop	4.71×10^{-3}	4.00×10^{-2}
5-6	8.91×10^{-1}	2.50×10^{-2}	stop	5.08×10^{-3}	7.26×10^{-2}
10-12	8.92×10^{-1}	5.00×10^{-2}	stop	-2.77×10^{-3}	3.99×10^{-2}

APPENDIX G
IF STATEMENT STATISTICS

A. Holm-Bonferroni tables: aggregated `IfStmt`

TABLE XXIII: Holm-Bonferroni correction for IfStmt.

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
2-7	$7.74 \times 10^{-3,158}$	4.76×10^{-4}	reject	8.17×10^{-1}	1.33×10^{-2}
5-7	$1.26 \times 10^{-2,974}$	4.81×10^{-4}	reject	6.40×10^{-1}	1.07×10^{-2}
7-11	$1.68 \times 10^{-2,604}$	4.85×10^{-4}	reject	-8.03×10^{-1}	1.44×10^{-2}
7-10	$1.32 \times 10^{-2,596}$	4.90×10^{-4}	reject	-7.98×10^{-1}	1.43×10^{-2}
7-12	$1.09 \times 10^{-2,537}$	4.95×10^{-4}	reject	-7.92×10^{-1}	1.44×10^{-2}
7-13	$6.74 \times 10^{-1,139}$	5.00×10^{-4}	reject	-6.65×10^{-1}	1.80×10^{-2}
7-9	$1.04 \times 10^{-1,098}$	5.05×10^{-4}	reject	-6.55×10^{-1}	1.81×10^{-2}
3-7	4.40×10^{-961}	5.10×10^{-4}	reject	7.64×10^{-1}	2.25×10^{-2}
4-7	1.02×10^{-749}	5.16×10^{-4}	reject	7.87×10^{-1}	2.63×10^{-2}
7-14	3.28×10^{-447}	5.21×10^{-4}	reject	-6.53×10^{-1}	2.83×10^{-2}
2-5	1.82×10^{-422}	5.26×10^{-4}	reject	1.76×10^{-1}	7.86×10^{-3}
1-7	8.75×10^{-410}	5.32×10^{-4}	reject	4.83×10^{-1}	2.19×10^{-2}
2-6	3.35×10^{-329}	5.38×10^{-4}	reject	3.99×10^{-1}	2.02×10^{-2}
7-8	5.83×10^{-326}	5.44×10^{-4}	reject	-6.52×10^{-1}	3.31×10^{-2}
6-7	1.11×10^{-318}	5.50×10^{-4}	reject	4.18×10^{-1}	2.14×10^{-2}
6-11	2.90×10^{-285}	5.56×10^{-4}	reject	-3.85×10^{-1}	2.09×10^{-2}
6-10	2.92×10^{-279}	5.62×10^{-4}	reject	-3.80×10^{-1}	2.09×10^{-2}
6-12	4.54×10^{-270}	5.68×10^{-4}	reject	-3.74×10^{-1}	2.09×10^{-2}
5-11	5.69×10^{-241}	5.75×10^{-4}	reject	-1.62×10^{-1}	9.59×10^{-3}
5-10	2.03×10^{-231}	5.81×10^{-4}	reject	-1.57×10^{-1}	9.49×10^{-3}
1-2	4.47×10^{-221}	5.88×10^{-4}	reject	-3.34×10^{-1}	2.06×10^{-2}
5-12	3.39×10^{-211}	5.95×10^{-4}	reject	-1.52×10^{-1}	9.59×10^{-3}
1-11	1.90×10^{-189}	6.02×10^{-4}	reject	-3.19×10^{-1}	2.13×10^{-2}
1-10	1.81×10^{-184}	6.10×10^{-4}	reject	-3.15×10^{-1}	2.13×10^{-2}
1-12	2.34×10^{-177}	6.17×10^{-4}	reject	-3.09×10^{-1}	2.13×10^{-2}
3-6	1.43×10^{-137}	6.25×10^{-4}	reject	3.46×10^{-1}	2.72×10^{-2}
4-6	1.26×10^{-125}	6.33×10^{-4}	reject	3.69×10^{-1}	3.04×10^{-2}
5-6	5.72×10^{-122}	6.41×10^{-4}	reject	2.23×10^{-1}	1.86×10^{-2}
6-13	5.42×10^{-94}	6.49×10^{-4}	reject	-2.47×10^{-1}	2.36×10^{-2}
1-3	5.39×10^{-89}	6.58×10^{-4}	reject	-2.81×10^{-1}	2.75×10^{-2}
6-9	1.76×10^{-86}	6.67×10^{-4}	reject	-2.37×10^{-1}	2.36×10^{-2}
1-4	4.60×10^{-84}	6.76×10^{-4}	reject	-3.04×10^{-1}	3.07×10^{-2}
2-9	3.96×10^{-82}	6.85×10^{-4}	reject	1.62×10^{-1}	1.65×10^{-2}
2-13	3.74×10^{-73}	6.94×10^{-4}	reject	1.52×10^{-1}	1.65×10^{-2}
9-11	5.79×10^{-62}	7.04×10^{-4}	reject	-1.48×10^{-1}	1.74×10^{-2}
1-5	8.49×10^{-59}	7.14×10^{-4}	reject	-1.57×10^{-1}	1.91×10^{-2}
9-10	2.30×10^{-58}	7.25×10^{-4}	reject	-1.43×10^{-1}	1.74×10^{-2}
11-13	1.58×10^{-54}	7.35×10^{-4}	reject	1.38×10^{-1}	1.74×10^{-2}
9-12	1.01×10^{-53}	7.46×10^{-4}	reject	-1.37×10^{-1}	1.74×10^{-2}
10-13	3.95×10^{-51}	7.58×10^{-4}	reject	1.33×10^{-1}	1.73×10^{-2}
1-13	4.37×10^{-50}	7.69×10^{-4}	reject	-1.82×10^{-1}	2.39×10^{-2}
6-14	7.24×10^{-47}	7.81×10^{-4}	reject	-2.35×10^{-1}	3.21×10^{-2}
12-13	8.39×10^{-47}	7.94×10^{-4}	reject	1.27×10^{-1}	1.74×10^{-2}
1-9	7.89×10^{-45}	8.07×10^{-4}	reject	-1.72×10^{-1}	2.40×10^{-2}
6-8	2.15×10^{-36}	8.20×10^{-4}	reject	-2.34×10^{-1}	3.64×10^{-2}
3-5	3.47×10^{-34}	8.33×10^{-4}	reject	1.23×10^{-1}	1.98×10^{-2}
4-5	5.60×10^{-33}	8.48×10^{-4}	reject	1.47×10^{-1}	2.40×10^{-2}
2-14	7.62×10^{-32}	8.62×10^{-4}	reject	1.64×10^{-1}	2.73×10^{-2}
11-14	7.85×10^{-26}	8.77×10^{-4}	reject	1.49×10^{-1}	2.79×10^{-2}
1-14	7.79×10^{-25}	8.93×10^{-4}	reject	-1.70×10^{-1}	3.24×10^{-2}
10-14	2.58×10^{-24}	9.09×10^{-4}	reject	1.45×10^{-1}	2.78×10^{-2}
2-8	1.09×10^{-23}	9.26×10^{-4}	reject	1.65×10^{-1}	3.23×10^{-2}
12-14	1.50×10^{-22}	9.43×10^{-4}	reject	1.39×10^{-1}	2.79×10^{-2}
4-9	3.10×10^{-20}	9.62×10^{-4}	reject	1.32×10^{-1}	2.81×10^{-2}
8-11	1.63×10^{-19}	9.80×10^{-4}	reject	-1.51×10^{-1}	3.27×10^{-2}
1-8	2.00×10^{-19}	1.00×10^{-3}	reject	-1.69×10^{-1}	3.66×10^{-2}
0-7	7.86×10^{-19}	1.02×10^{-3}	reject	5.63×10^{-1}	1.24×10^{-1}
8-10	2.17×10^{-18}	1.04×10^{-3}	reject	-1.46×10^{-1}	3.27×10^{-2}
3-9	4.40×10^{-18}	1.06×10^{-3}	reject	1.09×10^{-1}	2.46×10^{-2}
4-13	1.38×10^{-17}	1.09×10^{-3}	reject	1.22×10^{-1}	2.80×10^{-2}
8-12	4.20×10^{-17}	1.11×10^{-3}	reject	-1.40×10^{-1}	3.27×10^{-2}
3-13	2.93×10^{-15}	1.14×10^{-3}	reject	9.88×10^{-2}	2.45×10^{-2}
4-14	1.52×10^{-13}	1.16×10^{-3}	reject	1.34×10^{-1}	3.55×10^{-2}

Continued on next page

TABLE XXIII – continued from previous page

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
4-8	1.79×10^{-11}	1.19×10^{-3}	reject	1.35×10^{-1}	3.95×10^{-2}
3-14	4.10×10^{-11}	1.22×10^{-3}	reject	1.11×10^{-1}	3.28×10^{-2}
3-8	3.09×10^{-9}	1.25×10^{-3}	reject	1.12×10^{-1}	3.70×10^{-2}
2-3	1.01×10^{-6}	1.28×10^{-3}	reject	5.32×10^{-2}	2.13×10^{-2}
1-6	1.43×10^{-6}	1.32×10^{-3}	reject	6.55×10^{-2}	2.66×10^{-2}
0-2	6.12×10^{-5}	1.35×10^{-3}	reject	-2.54×10^{-1}	1.24×10^{-1}
2-12	8.65×10^{-5}	1.39×10^{-3}	reject	2.48×10^{-2}	1.24×10^{-2}
0-11	1.57×10^{-4}	1.43×10^{-3}	reject	-2.40×10^{-1}	1.24×10^{-1}
0-10	2.14×10^{-4}	1.47×10^{-3}	reject	-2.35×10^{-1}	1.24×10^{-1}
0-12	3.02×10^{-4}	1.52×10^{-3}	reject	-2.29×10^{-1}	1.24×10^{-1}
0-4	5.03×10^{-4}	1.56×10^{-3}	reject	-2.24×10^{-1}	1.26×10^{-1}
3-11	5.36×10^{-4}	1.61×10^{-3}	reject	-3.89×10^{-2}	2.20×10^{-2}
5-13	9.37×10^{-4}	1.67×10^{-3}	reject	-2.45×10^{-2}	1.45×10^{-2}
0-3	1.71×10^{-3}	1.72×10^{-3}	reject	-2.01×10^{-1}	1.26×10^{-1}
2-10	2.19×10^{-3}	1.79×10^{-3}	stop	1.92×10^{-2}	1.23×10^{-2}
3-10	2.46×10^{-3}	1.85×10^{-3}	stop	-3.39×10^{-2}	2.20×10^{-2}
3-12	1.15×10^{-2}	1.92×10^{-3}	stop	-2.84×10^{-2}	2.20×10^{-2}
2-4	2.04×10^{-2}	2.00×10^{-3}	stop	2.99×10^{-2}	2.53×10^{-2}
0-6	2.33×10^{-2}	2.08×10^{-3}	stop	1.45×10^{-1}	1.25×10^{-1}
2-11	2.37×10^{-2}	2.17×10^{-3}	stop	1.43×10^{-2}	1.24×10^{-2}
5-9	4.96×10^{-2}	2.27×10^{-3}	stop	-1.46×10^{-2}	1.46×10^{-2}
0-13	1.09×10^{-1}	2.38×10^{-3}	stop	-1.02×10^{-1}	1.25×10^{-1}
11-12	1.28×10^{-1}	2.50×10^{-3}	stop	1.05×10^{-2}	1.35×10^{-2}
3-4	1.43×10^{-1}	2.63×10^{-3}	stop	-2.33×10^{-2}	3.11×10^{-2}
0-9	1.48×10^{-1}	2.78×10^{-3}	stop	-9.22×10^{-2}	1.25×10^{-1}
0-14	1.62×10^{-1}	2.94×10^{-3}	stop	-9.03×10^{-2}	1.27×10^{-1}
0-8	1.73×10^{-1}	3.13×10^{-3}	stop	-8.89×10^{-2}	1.28×10^{-1}
0-1	2.13×10^{-1}	3.33×10^{-3}	stop	7.96×10^{-2}	1.25×10^{-1}
0-5	2.20×10^{-1}	3.57×10^{-3}	stop	-7.76×10^{-2}	1.24×10^{-1}
4-11	2.37×10^{-1}	3.85×10^{-3}	stop	-1.56×10^{-2}	2.59×10^{-2}
5-14	3.40×10^{-1}	4.17×10^{-3}	stop	-1.27×10^{-2}	2.62×10^{-2}
9-13	3.46×10^{-1}	4.55×10^{-3}	stop	-9.87×10^{-3}	2.05×10^{-2}
10-12	4.18×10^{-1}	5.00×10^{-3}	stop	5.56×10^{-3}	1.35×10^{-2}
4-10	4.18×10^{-1}	5.56×10^{-3}	stop	-1.07×10^{-2}	2.58×10^{-2}
13-14	4.43×10^{-1}	6.25×10^{-3}	stop	1.17×10^{-2}	2.99×10^{-2}
8-13	4.54×10^{-1}	7.14×10^{-3}	stop	-1.32×10^{-2}	3.45×10^{-2}
10-11	4.73×10^{-1}	8.33×10^{-3}	stop	-4.94×10^{-3}	1.35×10^{-2}
5-8	4.80×10^{-1}	1.00×10^{-2}	stop	-1.13×10^{-2}	3.13×10^{-2}
4-12	6.99×10^{-1}	1.25×10^{-2}	stop	-5.10×10^{-3}	2.58×10^{-2}
8-9	8.51×10^{-1}	1.67×10^{-2}	stop	-3.31×10^{-3}	3.45×10^{-2}
9-14	9.04×10^{-1}	2.50×10^{-2}	stop	1.85×10^{-3}	2.99×10^{-2}
8-14	9.44×10^{-1}	5.00×10^{-2}	stop	-1.47×10^{-3}	4.08×10^{-2}

APPENDIX H
WHILE STATEMENT STATISTICS

APPENDIX I
HOLM-BONFERRONI TABLES: AGGREGATED WHILESTMT

TABLE XXIV: Holm-Bonferroni correction for WhileStmt.

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
4-5	4.46×10^{-226}	1.11×10^{-3}	reject	3.96×10^{-1}	2.42×10^{-2}
3-5	1.18×10^{-166}	1.14×10^{-3}	reject	2.85×10^{-1}	2.03×10^{-2}
4-12	5.27×10^{-55}	1.16×10^{-3}	reject	3.97×10^{-1}	4.98×10^{-2}
4-9	1.66×10^{-38}	1.19×10^{-3}	reject	4.75×10^{-1}	7.17×10^{-2}
4-11	1.43×10^{-36}	1.22×10^{-3}	reject	3.22×10^{-1}	4.99×10^{-2}
4-10	6.57×10^{-34}	1.25×10^{-3}	reject	3.07×10^{-1}	4.95×10^{-2}
3-12	2.06×10^{-31}	1.28×10^{-3}	reject	2.86×10^{-1}	4.80×10^{-2}
3-9	5.09×10^{-24}	1.32×10^{-3}	reject	3.63×10^{-1}	7.05×10^{-2}
3-11	1.07×10^{-17}	1.35×10^{-3}	reject	2.10×10^{-1}	4.81×10^{-2}
4-13	1.12×10^{-17}	1.39×10^{-3}	reject	4.42×10^{-1}	1.01×10^{-1}
3-10	1.00×10^{-15}	1.43×10^{-3}	reject	1.96×10^{-1}	4.77×10^{-2}
4-6	5.19×10^{-13}	1.47×10^{-3}	reject	4.74×10^{-1}	1.29×10^{-1}
3-4	4.10×10^{-12}	1.52×10^{-3}	reject	-1.11×10^{-1}	3.14×10^{-2}
3-13	1.03×10^{-10}	1.56×10^{-3}	reject	3.31×10^{-1}	1.00×10^{-1}
3-6	2.75×10^{-8}	1.61×10^{-3}	reject	3.63×10^{-1}	1.28×10^{-1}
4-8	8.12×10^{-7}	1.67×10^{-3}	reject	2.65×10^{-1}	1.05×10^{-1}
9-10	4.07×10^{-5}	1.72×10^{-3}	reject	-1.68×10^{-1}	8.02×10^{-2}
5-10	4.92×10^{-5}	1.79×10^{-3}	reject	-8.97×10^{-2}	4.33×10^{-2}
9-11	1.91×10^{-4}	1.85×10^{-3}	reject	-1.53×10^{-1}	8.04×10^{-2}
5-11	7.94×10^{-4}	1.92×10^{-3}	reject	-7.49×10^{-2}	4.38×10^{-2}
8-9	8.03×10^{-4}	2.00×10^{-3}	reject	2.10×10^{-1}	1.23×10^{-1}
3-8	3.93×10^{-3}	2.08×10^{-3}	stop	1.54×10^{-1}	1.04×10^{-1}
10-12	3.98×10^{-3}	2.17×10^{-3}	stop	9.02×10^{-2}	6.14×10^{-2}
6-8	1.16×10^{-2}	2.27×10^{-3}	stop	-2.10×10^{-1}	1.63×10^{-1}
5-8	1.18×10^{-2}	2.38×10^{-3}	stop	-1.32×10^{-1}	1.02×10^{-1}
10-13	1.36×10^{-2}	2.50×10^{-3}	stop	1.35×10^{-1}	1.07×10^{-1}
6-10	1.40×10^{-2}	2.63×10^{-3}	stop	-1.68×10^{-1}	1.34×10^{-1}
8-13	1.45×10^{-2}	2.78×10^{-3}	stop	1.77×10^{-1}	1.42×10^{-1}
11-12	1.67×10^{-2}	2.94×10^{-3}	stop	7.54×10^{-2}	6.17×10^{-2}
8-12	2.00×10^{-2}	3.13×10^{-3}	stop	1.32×10^{-1}	1.11×10^{-1}
5-9	2.34×10^{-2}	3.33×10^{-3}	stop	7.82×10^{-2}	6.76×10^{-2}
6-11	2.52×10^{-2}	3.57×10^{-3}	stop	-1.53×10^{-1}	1.34×10^{-1}
11-13	2.82×10^{-2}	3.85×10^{-3}	stop	1.20×10^{-1}	1.08×10^{-1}
9-12	5.83×10^{-2}	4.17×10^{-3}	stop	-7.77×10^{-2}	8.04×10^{-2}
5-6	2.28×10^{-1}	4.55×10^{-3}	stop	7.79×10^{-2}	1.27×10^{-1}
6-12	2.57×10^{-1}	5.00×10^{-3}	stop	-7.74×10^{-2}	1.34×10^{-1}
8-11	3.18×10^{-1}	5.56×10^{-3}	stop	5.67×10^{-2}	1.11×10^{-1}
5-13	3.64×10^{-1}	6.25×10^{-3}	stop	4.55×10^{-2}	9.83×10^{-2}
12-13	4.12×10^{-1}	7.14×10^{-3}	stop	4.50×10^{-2}	1.08×10^{-1}
8-10	4.60×10^{-1}	8.33×10^{-3}	stop	4.19×10^{-2}	1.11×10^{-1}
9-13	5.91×10^{-1}	1.00×10^{-2}	stop	-3.27×10^{-2}	1.19×10^{-1}
10-11	6.36×10^{-1}	1.25×10^{-2}	stop	1.48×10^{-2}	6.15×10^{-2}
6-13	6.92×10^{-1}	1.67×10^{-2}	stop	-3.24×10^{-2}	1.60×10^{-1}
5-12	9.82×10^{-1}	2.50×10^{-2}	stop	5.00×10^{-4}	4.37×10^{-2}
6-9	9.97×10^{-1}	5.00×10^{-2}	stop	2.51×10^{-4}	1.43×10^{-1}

APPENDIX J
HOLM-BONFERRONI TABLES: AGGREGATED CALLEXPR

TABLE XXV: Holm-Bonferroni correction for CallExpr.

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
2-7	$1.54 \times 10^{-3,398}$	6.41×10^{-4}	reject	8.59×10^{-1}	1.35×10^{-2}
5-7	$2.73 \times 10^{-3,028}$	6.49×10^{-4}	reject	7.67×10^{-1}	1.27×10^{-2}
3-7	$3.28 \times 10^{-2,918}$	6.58×10^{-4}	reject	8.66×10^{-1}	1.47×10^{-2}
2-14	$3.77 \times 10^{-2,613}$	6.67×10^{-4}	reject	4.27×10^{-1}	7.64×10^{-3}
5-14	$1.79 \times 10^{-2,395}$	6.76×10^{-4}	reject	3.35×10^{-1}	6.25×10^{-3}
1-7	$4.46 \times 10^{-1,790}$	6.85×10^{-4}	reject	6.52×10^{-1}	1.41×10^{-2}
3-14	$3.50 \times 10^{-1,718}$	6.94×10^{-4}	reject	4.34×10^{-1}	9.58×10^{-3}
7-10	$2.06 \times 10^{-1,560}$	7.04×10^{-4}	reject	-7.37×10^{-1}	1.70×10^{-2}
7-11	$6.15 \times 10^{-1,560}$	7.14×10^{-4}	reject	-7.40×10^{-1}	1.71×10^{-2}
7-13	$1.72 \times 10^{-1,463}$	7.25×10^{-4}	reject	-7.98×10^{-1}	1.91×10^{-2}
7-12	$5.52 \times 10^{-1,437}$	7.35×10^{-4}	reject	-7.10×10^{-1}	1.71×10^{-2}
7-14	1.39×10^{-779}	7.46×10^{-4}	reject	-4.32×10^{-1}	1.42×10^{-2}
1-2	7.77×10^{-635}	7.58×10^{-4}	reject	-2.07×10^{-1}	7.52×10^{-3}
1-14	3.44×10^{-538}	7.69×10^{-4}	reject	2.20×10^{-1}	8.69×10^{-3}
13-14	4.61×10^{-467}	7.81×10^{-4}	reject	3.66×10^{-1}	1.55×10^{-2}
11-14	8.63×10^{-467}	7.94×10^{-4}	reject	3.08×10^{-1}	1.30×10^{-2}
10-14	2.53×10^{-464}	8.07×10^{-4}	reject	3.05×10^{-1}	1.29×10^{-2}
1-3	7.98×10^{-428}	8.20×10^{-4}	reject	-2.14×10^{-1}	9.48×10^{-3}
6-7	1.13×10^{-397}	8.33×10^{-4}	reject	6.69×10^{-1}	3.08×10^{-2}
7-9	1.95×10^{-394}	8.48×10^{-4}	reject	-5.38×10^{-1}	2.48×10^{-2}
12-14	6.33×10^{-381}	8.62×10^{-4}	reject	2.78×10^{-1}	1.30×10^{-2}
2-5	1.23×10^{-358}	8.77×10^{-4}	reject	9.24×10^{-2}	4.47×10^{-3}
1-5	5.67×10^{-296}	8.93×10^{-4}	reject	-1.14×10^{-1}	6.10×10^{-3}
7-8	1.52×10^{-282}	9.09×10^{-4}	reject	-5.05×10^{-1}	2.75×10^{-2}
2-9	3.15×10^{-183}	9.26×10^{-4}	reject	3.21×10^{-1}	2.18×10^{-2}
3-9	5.21×10^{-179}	9.43×10^{-4}	reject	3.28×10^{-1}	2.25×10^{-2}
2-8	4.42×10^{-172}	9.62×10^{-4}	reject	3.54×10^{-1}	2.48×10^{-2}
3-8	5.58×10^{-170}	9.80×10^{-4}	reject	3.62×10^{-1}	2.55×10^{-2}
3-5	2.97×10^{-157}	1.00×10^{-3}	reject	9.97×10^{-2}	7.31×10^{-3}
2-12	6.96×10^{-126}	1.02×10^{-3}	reject	1.50×10^{-1}	1.23×10^{-2}
3-12	1.98×10^{-113}	1.04×10^{-3}	reject	1.57×10^{-1}	1.36×10^{-2}
5-8	6.64×10^{-98}	1.06×10^{-3}	reject	2.62×10^{-1}	2.44×10^{-2}
5-9	9.98×10^{-98}	1.09×10^{-3}	reject	2.28×10^{-1}	2.13×10^{-2}
8-13	5.14×10^{-92}	1.11×10^{-3}	reject	-2.93×10^{-1}	2.83×10^{-2}
9-13	6.29×10^{-88}	1.14×10^{-3}	reject	-2.60×10^{-1}	2.56×10^{-2}
2-10	1.26×10^{-86}	1.16×10^{-3}	reject	1.23×10^{-1}	1.22×10^{-2}
2-11	6.27×10^{-81}	1.19×10^{-3}	reject	1.19×10^{-1}	1.23×10^{-2}
3-10	1.95×10^{-79}	1.22×10^{-3}	reject	1.30×10^{-1}	1.35×10^{-2}
1-13	1.59×10^{-76}	1.25×10^{-3}	reject	-1.46×10^{-1}	1.55×10^{-2}
3-11	1.16×10^{-74}	1.28×10^{-3}	reject	1.27×10^{-1}	1.36×10^{-2}
8-11	3.21×10^{-65}	1.32×10^{-3}	reject	-2.35×10^{-1}	2.70×10^{-2}
8-10	9.61×10^{-64}	1.35×10^{-3}	reject	-2.32×10^{-1}	2.69×10^{-2}
9-11	9.78×10^{-60}	1.39×10^{-3}	reject	-2.01×10^{-1}	2.42×10^{-2}
6-14	3.52×10^{-59}	1.43×10^{-3}	reject	2.37×10^{-1}	2.87×10^{-2}
9-10	3.53×10^{-58}	1.47×10^{-3}	reject	-1.98×10^{-1}	2.42×10^{-2}
8-12	4.88×10^{-50}	1.52×10^{-3}	reject	-2.05×10^{-1}	2.70×10^{-2}
9-12	9.71×10^{-44}	1.56×10^{-3}	reject	-1.71×10^{-1}	2.42×10^{-2}
3-6	1.37×10^{-40}	1.61×10^{-3}	reject	1.97×10^{-1}	2.89×10^{-2}
1-11	7.35×10^{-40}	1.67×10^{-3}	reject	-8.74×10^{-2}	1.30×10^{-2}
2-6	2.72×10^{-39}	1.72×10^{-3}	reject	1.90×10^{-1}	2.84×10^{-2}
1-10	1.05×10^{-37}	1.79×10^{-3}	reject	-8.43×10^{-2}	1.29×10^{-2}
1-8	1.75×10^{-30}	1.85×10^{-3}	reject	1.47×10^{-1}	2.52×10^{-2}
1-9	7.48×10^{-24}	1.92×10^{-3}	reject	1.14×10^{-1}	2.22×10^{-2}
5-12	1.73×10^{-22}	2.00×10^{-3}	reject	5.71×10^{-2}	1.15×10^{-2}
12-13	1.75×10^{-21}	2.08×10^{-3}	reject	-8.87×10^{-2}	1.83×10^{-2}
9-14	6.85×10^{-21}	2.17×10^{-3}	reject	1.06×10^{-1}	2.22×10^{-2}
6-8	3.92×10^{-18}	2.27×10^{-3}	reject	1.65×10^{-1}	3.72×10^{-2}
1-12	4.12×10^{-18}	2.38×10^{-3}	reject	-5.74×10^{-2}	1.30×10^{-2}
3-13	6.78×10^{-17}	2.50×10^{-3}	reject	6.81×10^{-2}	1.60×10^{-2}
6-13	8.89×10^{-16}	2.63×10^{-3}	reject	-1.29×10^{-1}	3.14×10^{-2}
2-13	1.17×10^{-15}	2.78×10^{-3}	reject	6.09×10^{-2}	1.49×10^{-2}
6-9	2.84×10^{-13}	2.94×10^{-3}	reject	1.31×10^{-1}	3.52×10^{-2}
5-6	9.99×10^{-12}	3.13×10^{-3}	reject	9.73×10^{-2}	2.80×10^{-2}
10-13	2.84×10^{-11}	3.33×10^{-3}	reject	-6.18×10^{-2}	1.82×10^{-2}
11-13	3.19×10^{-10}	3.57×10^{-3}	reject	-5.86×10^{-2}	1.83×10^{-2}

Continued on next page

TABLE XXV – continued from previous page

Comparison	P-value	Holm Threshold	Decision	CI Estimate	CI Error
8-14	1.52×10^{-8}	3.85×10^{-3}	reject	7.28×10^{-2}	2.52×10^{-2}
5-10	1.93×10^{-7}	4.17×10^{-3}	reject	3.02×10^{-2}	1.14×10^{-2}
5-11	3.94×10^{-6}	4.55×10^{-3}	reject	2.70×10^{-2}	1.15×10^{-2}
6-11	5.36×10^{-6}	5.00×10^{-3}	reject	-7.02×10^{-2}	3.03×10^{-2}
6-10	1.35×10^{-5}	5.56×10^{-3}	reject	-6.71×10^{-2}	3.02×10^{-2}
5-13	1.37×10^{-5}	6.25×10^{-3}	reject	-3.16×10^{-2}	1.42×10^{-2}
11-12	2.76×10^{-4}	7.14×10^{-3}	reject	3.01×10^{-2}	1.62×10^{-2}
10-12	1.07×10^{-3}	8.33×10^{-3}	reject	2.69×10^{-2}	1.61×10^{-2}
6-12	9.24×10^{-3}	1.00×10^{-2}	reject	-4.02×10^{-2}	3.03×10^{-2}
8-9	4.31×10^{-2}	1.25×10^{-2}	stop	-3.35×10^{-2}	3.24×10^{-2}
2-3	9.68×10^{-2}	1.67×10^{-2}	stop	-7.22×10^{-3}	8.53×10^{-3}
1-6	2.40×10^{-1}	2.50×10^{-2}	stop	-1.72×10^{-2}	2.87×10^{-2}
10-11	7.01×10^{-1}	5.00×10^{-2}	stop	-3.17×10^{-3}	1.61×10^{-2}